

LẬP TRÌNH WEB

Chương 4 PHP Và MySQL

GV: Bùi Văn Thương

Email: thuong.bui@ut.edu.vn

CÔNG CỤ HỌC TẬP

Code editor:
Microsoft Visual
Studio Code



Môi trường ảo:
XAMPP (Windows)
hoặc MAMP (Mac)



Phần 1. PHP

PHP (PHP: Hypertext Preprocessor) là một ngôn ngữ lập trình kịch bản phía máy chủ (server-side scripting language) mã nguồn mở, được sử dụng chủ yếu để phát triển các ứng dụng web động, dễ dàng nhúng vào HTML, kết nối với cơ sở dữ liệu và xử lý dữ liệu, tạo ra các trang web tương tác và linh hoạt như WordPress, Drupal. Nó hoạt động ở server, sinh ra HTML để gửi về trình duyệt người dùng, giúp tạo nội dung động và xử lý các tác vụ phía máy chủ

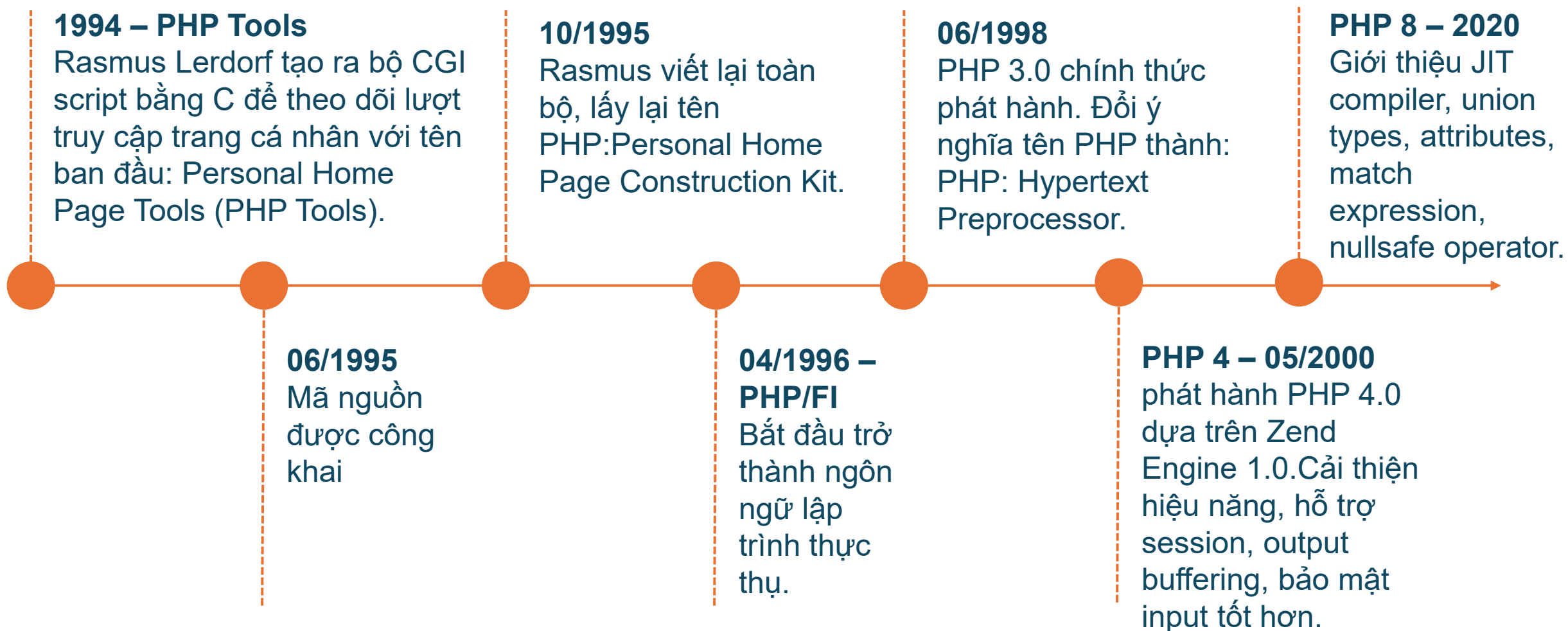
NỘI DUNG CHÍNH

KIẾN THỨC PHP
CĂN BẢN

CÁC ĐỐI TƯỢNG
QUAN TRỌNG
TRONG PHP

LẬP TRÌNH HƯỚNG
ĐỐI TƯỢNG TRONG
PHP

Lịch sử hình thành và phát triển của PHP:



Một số framework PHP phổ biến:

- Laravel: Cú pháp rõ ràng, tài liệu đầy đủ, cộng đồng lớn. Phù hợp với các dự án từ nhỏ đến lớn, đặc biệt là các ứng dụng web phức tạp
- Symfony: Cấu trúc module linh hoạt, độ ổn định cao. Hỗ trợ tái sử dụng component, tích hợp tốt với các dự án lớn. Phù hợp với hệ thống doanh nghiệp, dự án cần khả năng mở rộng.
- CodeIgniter: Nhẹ, tốc độ nhanh, dễ học. Không yêu cầu cấu hình phức tạp, dung lượng nhỏ. Phù hợp với dự án nhỏ, API nhanh gọn.
- Zend Framework / Laminas: Hướng đối tượng mạnh mẽ, linh hoạt. Tích hợp tốt với các hệ thống doanh nghiệp. Phù hợp: Ứng dụng quy mô lớn, yêu cầu bảo mật cao.
- Yii Framework: Hiệu suất cao, hỗ trợ Gii code generator, bảo mật tốt, hỗ trợ RESTful API. Phù hợp với ứng dụng web cần tốc độ xử lý nhanh.

Một số framework PHP phổ biến:

- CakePHP: Cấu trúc MVC rõ ràng, hỗ trợ CRUD nhanh. Bảo mật tốt, code generator tiện lợi. Phù hợp với dự án vừa và nhỏ.
- Slim Framework: Nhẹ, đơn giản, tối ưu cho API. Routing nhanh, dễ mở rộng. Phù hợp với xây dựng RESTful API, microservices.
- FuelPHP: Hỗ trợ cả MVC và HMVC, bảo mật tốt, linh hoạt. Phù hợp với dự án cần cấu trúc module rõ ràng.
- Laminas Project (kế thừa Zend): Hỗ trợ lâu dài, tiêu chuẩn cao. Phù hợp với ứng dụng doanh nghiệp, hệ thống lớn.
- Phalcon: Viết bằng C, tốc độ xử lý cực nhanh, tiêu tốn ít tài nguyên server. Phù hợp với hệ thống yêu cầu hiệu năng cao.

Một số CMS PHP phổ biến:

CMS là các bộ mã nguồn mở viết bằng PHP đã được dựng sẵn khung sườn (quản lý bài viết, người dùng, giao diện, bảo mật...). Việc của bạn là cài đặt, cấu hình và tùy biến theo ý khách hàng.

- WordPress: Chiếm hơn 40% thị phần website toàn cầu. Nếu bạn muốn làm blog, trang tin tức, hay web giới thiệu công ty nhanh gọn, đây là lựa chọn số 1.
- Magento: Phù hợp với web thương mại điện tử (E-commerce). Những trang bán hàng quy mô lớn với hàng nghìn sản phẩm thường dùng Magento.
- Drupal: Sự lựa chọn cho Chính phủ và Doanh nghiệp. Bảo mật cực cao, quản lý phân quyền người dùng chi tiết.
- Joomla: Quản lý nội dung phân cấp tốt hơn WordPress nhưng không quá khó như Drupal. Phù hợp cho các trang web cộng đồng hoặc mạng xã hội nội bộ.

Cài đặt môi trường lập trình PHP:

Tải và cài đặt XAMPP tại: <https://www.apachefriends.org/download.html>

XAMPP là một phần mềm mã nguồn mở miễn phí, tích hợp sẵn các thành phần cần thiết để tạo web server cục bộ (Localhost) trên máy tính.

Thành phần chính của XAMPP:

- Apache: Web server phổ biến nhất hiện nay.
- MariaDB/MySQL: Hệ quản trị cơ sở dữ liệu.
- PHP: Ngôn ngữ lập trình phía server.
- Perl: Ngôn ngữ lập trình đa năng.
- phpMyAdmin: Giao diện quản lý cơ sở dữ liệu MySQL/MariaDB.

Cài đặt môi trường lập trình PHP:

Cách hoạt động của Localhost qua XAMPP Khi cài đặt XAMPP, máy tính sẽ đóng vai trò như một máy chủ web với địa chỉ `http://localhost` hoặc `http://127.0.0.1`. Bạn có thể chạy mã PHP, quản lý cơ sở dữ liệu và thử nghiệm website ngay trên máy mà không cần internet.

Các bước chạy file PHP trên XAMPP:

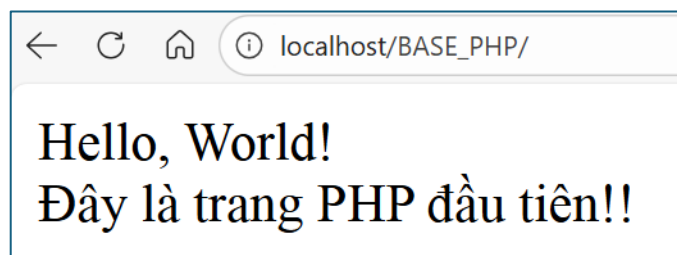
- Vào thư mục cài đặt của XAMPP, chuột phải vào file **xampp-control.exe** và chọn **Run as administrator**.
- Bấm Start ở dòng Apache.
- Lên trình duyệt mở đường dẫn **http://localhost** và xem kết quả. Toàn bộ tài nguyên hiển thị nằm trong thư mục **htdocs/** của Xampp.

Tạo file PHP đầu tiên:

- Trong thư mục htdocs tạo thư mục BASE_PHP
- Trong BASE_PHP tạo file **index.php** và soạn nội dung cho file **index.php** bằng trình soạn thảo code MS Visual Studio Code với nội dung:

```
<?php
    echo "Hello, World!";
    echo "<br>Đây là trang PHP đầu tiên!!";
?>
```

- Xem kết quả trên trình duyệt với đường dẫn http://localhost/BASE_PHP/



Chú ý: file index.php sẽ được chạy mặc định trong thư mục chứa nó

Cú pháp của PHP:

- File PHP thường chứa một số thẻ HTML và một số mã kịch bản PHP với phần mở rộng là .php.
- Các lệnh PHP được thực thi trên máy chủ và trả về kết quả HTML thuần túy.
- Ta có thể đặt tập lệnh PHP ở bất kỳ vị trí nào trong tài liệu với bắt đầu và kết thúc bởi cặp thẻ `<?php` và `?>`

```
<!DOCTYPE html>
<html>
<body>
    <h1>Học PHP cơ bản</h1>
    <?php
        echo 'Cú pháp PHP rất đơn giản!';
    ?>
</body>
</html>
```



Biến và kiểu dữ liệu:

Biến trong PHP luôn bắt đầu bằng dấu đô la \$ và có phân biệt chữ hoa thường.

```
<?php
    $name = "Nguyễn văn A"; // Kiểu Chuỗi (String)
    $age = 25;                // Kiểu Số nguyên (Integer)
    $is_learning = true;      // Kiểu Boolean (Đúng/Sai)
    $price = 10.5;            // Kiểu Số thực (Float)
    echo "Xin chào $name, bạn $age tuổi và đang học PHP cơ bản.";
?>
```

Xin chào Nguyễn văn A, bạn 25 tuổi và đang học PHP cơ bản.

- echo là lệnh xuất dữ liệu ra màn hình.
- Dấu chấm phẩy (;): Bắt buộc phải có ở cuối mỗi dòng lệnh.

Phân biệt ngoặc đơn và ngoặc kép:

Ngoặc đơn và kép đều dùng để biểu diễn kiểu chuỗi tùy nhiên:

- Ngoặc đơn: Hiển thị y hệt những gì bạn viết bên trong.
- Ngoặc kép: Có thể truyền giá trị của biến vào bên trong.

```
<?php
    $name = "Nguyễn văn A";
    echo "Xin chào, tôi tên là $name";
    echo "<br>"; // Xuống dòng
    echo '$name là một biến';
?>
```

Xin chào, tôi tên là Nguyễn văn A
\$name là một biến

Hằng trong PHP:

Trong PHP, chúng ta có 2 cách chính để tạo hằng số là dùng hàm define() hoặc từ khóa const

```
<?php
    define("APP_NAME", "PHP Tutorial");
    const DB_NAME = "lap_trinh_php";
    echo APP_NAME;
    echo " với CSDL ";
    echo DB_NAME;
?>
```

PHP Tutorial với CSDL lap_trinh_php

Ghi chú: Mặc dù PHP không bắt buộc, nhưng các lập trình viên chuyên nghiệp luôn đặt tên hằng số bằng **CHỮ HOA** và dùng dấu gạch dưới để ngăn cách các từ. Điều này giúp dễ dàng nhận diện hằng.

Hằng trong PHP:

Danh sách một số hằng hệ thống.

Hằng số	Mô tả chi tiết
__CLASS__	Trả về tên của lớp (Class) đang chứa nó (Dùng trong OOP).
__DIR__	Trả về đường dẫn thư mục chứa file đó. (Thay thế cho <code>dirname(__FILE__)</code>).
__FILE__	Trả về đường dẫn đầy đủ và tên file hiện tại.
__FUNCTION__	Trả về tên của hàm đang chứa hằng số này.
__LINE__	Trả về số dòng hiện tại của file code. Cực kỳ hữu ích khi ghi log lỗi.
__METHOD__	Trả về tên phương thức của lớp (bao gồm cả tên lớp, ví dụ: <code>MyClass::myMethod</code>).
__NAMESPACE__	Trả về tên không gian tên (namespace) hiện tại.

Toán tử trong PHP:

Toán tử	Mô tả chi tiết
+, -, *, /, %, **	Các toán tử số học: cộng, trừ, nhân, chia, chia lấy dư, lũy thừa
=, +=, -=, *=, /=, %=	Các toán tử gán
==, ===, !=, <>, !==	Các toán tử so sánh bằng và khác
>, <, <=, >=	Các toán tử so sánh lớn bé
<=>	Toán tử Spaceship trả về một số nguyên dựa trên kết quả so sánh giữa hai vế a và b (trả về 0 nếu a=b, 1 nếu a>b, -1 nếu a<b)
++, --	Toán tử tăng giảm 1
and, or, xor, &&, , !	Các toán tử quan hệ ở dạng chữ và ký hiệu

Toán tử trong PHP:

Một số toán tử đặc biệt khác:

Toán tử	Mô tả chi tiết	Ví dụ
?:	Toán tử 3 ngôi	<pre>\$diem = 8; \$ket_qua = (\$diem >= 5) ? "Đỗ" : "Trượt"; echo \$ket_qua; // Kết quả: Đỗ</pre>
.	Toán tử nối chuỗi	<pre>\$name = "UTH"; echo "Chào " . \$name . "!"; // Kết quả: Chào UTH!</pre>
??	Toán tử Null Coalescing	<pre>// Giả sử \$_GET['page'] không hề tồn tại trên URL \$page = \$_GET['page'] ?? 1; echo \$page; // Kết quả là 1</pre>
(kiểu dữ liệu)	Ép kiểu	<pre>\$string_num = "100"; \$int_num = (int)\$string_num;</pre>

Các cấu trúc điều khiển trong PHP:

PHP có đầy đủ các cấu trúc lệnh và hoạt động tương tự như các ngôn ngữ lập trình thông dụng khác:

- Cấu trúc rẽ nhánh: if...else, switch case.
- Cấu trúc lặp: for, do...while, while.
- Các lệnh nhảy: break, continue.

Mảng trong PHP:

Mảng trong PHP gồm mảng tuần tự và mảng kết hợp.

Mảng tuần tự

```
// Cách cũ (dùng hàm array)
$colors = array("Đỏ", "Xanh", "Vàng");

// Cách mới (dùng ngoặc vuông)
$names = ["An", "Bình", "Chi"];
echo $names[0]; // Kết quả: An
echo $names[2]; // Kết quả: Chi
echo $colors[1]; // Kết quả: Xanh
```

Mảng kết hợp

```
$sinh_vien = [
    "ho_ten" => "Nguyễn Văn An",
    "tuoi"   => 20,
    "lop"    => "PHP01"
];
echo "Họ tên: ".$sinh_vien["ho_ten"]."<br>";
echo "Tuổi: ".$sinh_vien["tuoi"]."<br>";
echo "Lớp: ".$sinh_vien["lop"]."<br>";
```

Họ tên: Nguyễn Văn An
Tuổi: 20
Lớp: PHP01

Mảng trong PHP:

Duyệt mảng bằng vòng lặp foreach():

Trong PHP, foreach là vòng lặp chuyên dụng và mạnh mẽ nhất để duyệt qua các Mảng (Array) hoặc Đối tượng (Object). Nó giúp ta lấy từng phần tử ra mà không cần quan tâm mảng đó dài bao nhiêu hay chỉ số (index) của nó là gì.

Ví dụ: foreach chỉ lấy giá trị (value)

```
$fruits = ["Táo", "Chuối", "Cam"];  
  
foreach ($fruits as $fruit) {  
    echo "Tôi thích ăn: $fruit <br>";  
}
```

```
Tôi thích ăn: Táo  
Tôi thích ăn: Chuối  
Tôi thích ăn: Cam
```

Mảng trong PHP:

Ví dụ: foreach lấy cả khóa (key) và giá trị (value)

```
$user = [  
    "Họ tên" => "Nguyễn Văn An",  
    "Lớp" => "PHP01",  
    "Điểm" => 9.5  
];  
  
foreach ($user as $label => $info) {  
    echo "$label: $info <br>";  
}
```

Họ tên: Nguyễn Văn An
Lớp: PHP01
Điểm: 9.5

Mảng trong PHP:

Hiển thị cấu trúc mảng với lệnh `print_r()` kết hợp thẻ `<pre>`

```
$user = [  
    "Họ tên" => "Nguyễn Văn An",  
    "Lớp" => "PHP01",  
    "Điểm" => 9.5  
];  
  
echo '<pre>';  
print_r($user);  
echo '</pre>';
```



```
Array  
(  
    [Họ tên] => Nguyễn Văn An  
    [Lớp] => PHP01  
    [Điểm] => 9.5  
)
```

Hàm trong PHP:

Cú pháp chung của hàm:

```
// 1. Định nghĩa hàm
```

```
function tenHam($param1, $param2, $param3="Giá trị mặc định") {  
    echo "Phần nội dung trong hàm!";  
    echo "<br>";  
    echo "Tham số 1: $param1";  
    echo "<br>";  
    echo "Tham số 2: $param2";  
    echo "<br>";  
    echo "Tham số 3: $param3";  
}
```

```
// 2. Gọi hàm
```

```
echo "Gọi hàm lần 1 với tham số mặc định<br>";  
tenHam("Giá trị 1", "Giá trị 2");  
echo "<br>Gọi hàm lần 2 với tham số tùy chỉnh<br>";  
tenHam("Giá trị 1", "Giá trị 2", "Giá trị 3");
```

Gọi hàm lần 1 với tham số mặc định

Phần nội dung trong hàm!

Tham số 1: Giá trị 1

Tham số 2: Giá trị 2

Tham số 3: Giá trị mặc định

Gọi hàm lần 2 với tham số tùy chỉnh

Phần nội dung trong hàm!

Tham số 1: Giá trị 1

Tham số 2: Giá trị 2

Tham số 3: Giá trị 3

Hàm trong PHP:

Giá trị trả về của hàm:

```
function tinhTong($a, $b) {  
    $tong = $a + $b;  
    return $tong; // Trả kết quả về  
}  
  
$ketQua = tinhTong(5, 10);  
echo "Tổng là: " . ($ketQua * 2);
```

Tổng là: 30

Ghi chú:

Tên hàm không phân biệt hoa thường, nhưng tốt nhất ta nên đặt tên theo quy tắc camelCase (ví dụ: tinhTong, guiEmail) để code chuyên nghiệp.

Phạm vi của biến trong hàm (Variable Scope):

Một lỗi kinh điển của các bạn mới học là cố gắng dùng biến bên ngoài vào trong hàm.

- Biến cục bộ: Biến khai báo trong hàm chỉ có giá trị trong hàm đó.
- Biến toàn cục: Muốn dùng biến bên ngoài vào trong hàm, ta phải dùng từ khóa global.

```
$age = 20;

function checkAge() {
    global $age; // Gọi biến bên ngoài vào
    echo "Tuổi của bạn là: ".$age;
}

checkAge(); // Gọi hàm để hiển thị tuổi
```

Tuổi của bạn là: 20

Hàm hệ thống trong PHP:

Nhóm hàm xử lý chuỗi:

- `strlen($str)` – Độ dài chuỗi (tính theo byte).
- `mb_strlen($str, 'UTF-8')` – Độ dài chuỗi (hỗ trợ Unicode).
- `substr_count($haystack, $needle)` – Đếm số lần xuất hiện của chuỗi con.
- `strpos($haystack, $needle)` – Vị trí xuất hiện đầu tiên.
- `strrpos($haystack, $needle)` – Vị trí xuất hiện cuối cùng.
- `substr($str, $start, $length)` – Cắt chuỗi.
- `explode($delimiter, $string)` – Tách chuỗi thành mảng.
- `str_split($string, $length)` – Chia chuỗi thành mảng con.
- `chunk_split($string, $length, $end)` – Chia chuỗi thành đoạn kèm ký tự kết thúc.

Hàm hệ thống trong PHP:

Nhóm hàm xử lý chuỗi:

- `str_replace($search, $replace, $subject)` – Thay thế chuỗi.
- `str_ireplace()` – Thay thế không phân biệt hoa thường.
- `substr_replace($string, $replacement, $start, $length)` – Thay thế một phần chuỗi.
- `preg_replace($pattern, $replacement, $subject)` – Thay thế bằng regex.
- `strtolower($str)` – Chuyển thành chữ thường.
- `strtoupper($str)` – Chuyển thành chữ hoa.
- `ucfirst($str)` – Viết hoa chữ cái đầu tiên.
- `ucwords($str)` – Viết hoa chữ cái đầu mỗi từ.

Hàm hệ thống trong PHP:

Nhóm hàm xử lý chuỗi:

- `trim($str)` – Xóa khoảng trắng đầu và cuối.
- `ltrim($str)` – Xóa khoảng trắng bên trái.
- `rtrim($str)` – Xóa khoảng trắng bên phải.
- `htmlspecialchars($str)` – Chuyển ký tự HTML đặc biệt thành dạng an toàn.
- `addslashes($str)` – Thêm dấu gạch chéo trước ký tự đặc biệt.
- `strcmp($str1, $str2)` – So sánh phân biệt hoa thường.
- `strcasecmp($str1, $str2)` – So sánh không phân biệt hoa thường.
- `strncmp($str1, $str2, $len)` – So sánh theo số ký tự.

Hàm hệ thống trong PHP:

Nhóm hàm mã hóa và xử lý đặc biệt:

- `md5($str)` – Mã hóa MD5.
- `sha1($str)` – Mã hóa SHA1.
- `base64_encode($str)` / `base64_decode($str)` – Mã hóa/giải mã Base64.
- `urlencode($str)` / `urldecode($str)` – Mã hóa/giải mã URL.
- `password_hash($password, PASSWORD_DEFAULT)`; Mã hóa mật khẩu.
- `password_verify($passwordInput, $passwordHash)`: Xác minh mật khẩu đã mã hóa bằng `password_hash`.

Hàm hệ thống trong PHP:

Nhóm hàm xử lý mảng:

- `array_fill()` – Tạo mảng với giá trị mặc định.
- `array_fill_keys()` – Tạo mảng với các key cho trước.
- `array_combine()` – Kết hợp 2 mảng thành mảng key–value.
- `array_merge()` – Gộp nhiều mảng.
- `array_replace()` – Thay thế giá trị mảng theo key.

```
$a = array_fill(0, 3, 'PHP'); // [PHP, PHP, PHP]  
$b = array_combine(['a', 'b'], [1, 2]); // ['a'=>1, 'b'=>2]
```

Hàm hệ thống trong PHP:

Nhóm hàm xử lý mảng:

- `in_array()` – Kiểm tra giá trị có tồn tại trong mảng.
- `array_key_exists()` – Kiểm tra key có tồn tại.
- `isset()` – Kiểm tra biến hoặc phần tử tồn tại và khác null.
- `array_search()` – Tìm key của giá trị.

```
$arr = ['a'=>1, 'b'=>2];  
if (array_key_exists('a', $arr)) echo "Có key a";  
  
$colors = ["Đỏ", "Xanh", "Vàng", "Tím"];  
$index = array_search("Vàng", $colors);  
echo "<br>Màu vàng nằm ở vị trí thứ: " . $index;
```

Có key a
Màu vàng nằm ở vị trí thứ: 2

Hàm hệ thống trong PHP:

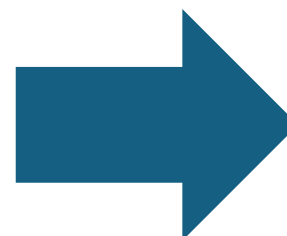
Nhóm hàm xử lý mảng:

- `sort()` – Sắp xếp tăng theo giá trị (mất key).
- `rsort()` – Sắp xếp giảm theo giá trị (mất key).
- `asort()` – Sắp xếp tăng theo giá trị (giữ key).
- `arsort()` – Sắp xếp giảm theo giá trị (giữ key).
- `krsort()` – Sắp xếp tăng theo key.
- `krsort()` – Sắp xếp giảm theo key.
- `usort()` – Sắp xếp theo hàm so sánh tùy chỉnh.

Hàm hệ thống trong PHP:

Ví dụ về sắp xếp mảng:

```
$arr1 = ["a"=>"3", "b"=>"1", "c"=>"2"];  
$arr2 = ["A"=>"3", "B"=>"1", "C"=>"2"];  
  
asort($arr1);  
echo '<pre>';  
print_r($arr1);  
echo '</pre>';  
  
sort($arr2);  
echo '<pre>';  
print_r($arr2);  
echo '</pre>';
```



```
Array  
(  
    [b] => 1  
    [c] => 2  
    [a] => 3  
)  
  
Array  
(  
    [0] => 1  
    [1] => 2  
    [2] => 3  
)
```

Hàm hệ thống trong PHP:

Hàm `array_filter()` lọc phần tử mảng theo điều kiện:

```
$user = [  
    "Nguyễn Văn An" => 8,  
    "Trần Văn Nhật" => 8.7,  
    "Lê Văn Hòa" => 9.5,  
    "Phạm Văn Hùng" => 7.5,  
    "Đỗ Văn Huy" => 6.5  
];  
  
$user_good = array_filter($user, function($value) {  
    return $value > 8;  
});  
echo "Danh sách học sinh có điểm trên 8: <br>";  
foreach ($user_good as $name => $score) {  
    echo "$name: $score <br>";  
}
```

Danh sách học sinh có điểm trên 8:
Trần Văn Nhật: 8.7
Lê Văn Hòa: 9.5

Hàm hệ thống trong PHP:

Hàm `array_filter()` lọc phần tử mảng theo điều kiện:

- Nếu ta không truyền hàm callback vào, `array_filter()` sẽ tự động loại bỏ tất cả các phần tử có giá trị tương đương với false (như `null`, `0`, `""`, `[]`).
- Một điểm đặc biệt là `array_filter()` giữ nguyên các khóa ban đầu của mảng.

```
$data = ["An", 0, null, "Bình", "", false, "Chi"];

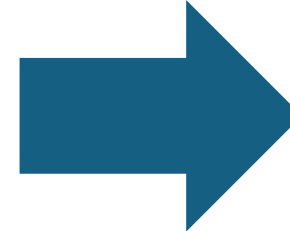
$cleanData = array_filter($data);
echo "<pre>";
print_r($cleanData);
echo "</pre>";
```

```
Array
(
    [0] => An
    [3] => Bình
    [6] => Chi
)
```

Hàm hệ thống trong PHP:

Hàm `array_map()` áp dụng hàm cho từng phần tử:

```
$user = [  
    "Nguyễn Văn An" => 4,  
    "Trần Văn Nhật" => 7.7,  
    "Lê Văn Hòa" => 8.5,  
    "Phạm Văn Hùng" => 6.5,  
    "Đỗ Văn Huy" => 4.5  
];  
  
$user_pass = array_map(function($value) {  
    return $value >= 5 ? "Đậu" : "Rớt";  
}, $user);  
echo "Danh sách kết quả tuyển sinh: <br>";  
foreach ($user_pass as $name => $status) {  
    echo "<p>$name: $status </p>";  
}
```



Danh sách kết quả tuyển sinh:

Nguyễn Văn An: Rớt

Trần Văn Nhật: Đậu

Lê Văn Hòa: Đậu

Phạm Văn Hùng: Đậu

Đỗ Văn Huy: Rớt

Hàm hệ thống trong PHP:

`array_map()` có thể xử lý 2, 3 mảng cùng lúc:

```
$foods = ["Phở", "Bánh mì", "Xôi"];  
$prices = [50000, 20000, 15000];  
  
$menu = array_map(function($f, $p) {  
    return "$f có giá $p VNĐ";  
}, $foods, $prices);  
  
echo "<pre>";  
print_r($menu);  
echo "</pre>";
```

```
Array  
(  
    [0] => Phở có giá 50000 VNĐ  
    [1] => Bánh mì có giá 20000 VNĐ  
    [2] => Xôi có giá 15000 VNĐ  
)
```

Hàm hệ thống trong PHP:

array_walk() – Duyệt và xử lý từng phần tử:

```
$fruits = ["a" => "Táo", "b" => "Chuối", "c" => "Cam"];  
  
array_walk($fruits, function($value, $key) {  
    echo "Ký hiệu: $key có giá trị là $value <br>";  
});
```

Ký hiệu: a có giá trị là Táo
Ký hiệu: b có giá trị là Chuối
Ký hiệu: c có giá trị là Cam

Ta dùng dấu & ở tham số \$value để thay đổi mảng gốc.

```
$prices = [100, 200, 300];  
array_walk($prices, function(&$value) {  
    $value = $value . " VNĐ";  
});  
echo "<pre>";  
print_r($prices);  
echo "</pre>";
```

```
Array  
(  
    [0] => 100 VNĐ  
    [1] => 200 VNĐ  
    [2] => 300 VNĐ  
)
```

Hàm hệ thống trong PHP:

Sử dụng tham số bổ sung trong array_walk():

```
$users = ["An", "Binh", "Chi"];  
$prefix = "Sinh viên: ";  
  
array_walk($users, function(&$value, $key, $p) {  
    $value = $p . $value;  
}, $prefix);  
  
echo "<pre>";  
print_r($users);  
echo "</pre>";
```

```
Array  
(  
    [0] => Sinh viên: An  
    [1] => Sinh viên: Binh  
    [2] => Sinh viên: Chi  
)
```

Hàm hệ thống trong PHP:

`array_reduce()` – Gộp mảng thành một giá trị duy nhất.

```
$numbers = [1, 2, 3, 4, 5];  
  
$sum = array_reduce($numbers, function($carry, $item) {  
    return $carry + $item;  
}, 0); // Bắt đầu từ 0  
  
echo $sum; // Kết quả: 15
```

Cơ chế chạy:

- Lần 1: `$carry = 0`, `$item = 1` => Trả về 1.
- Lần 2: `$carry = 1`, `$item = 2` => Trả về 3.
- Lần 3: `$carry = 3`, `$item = 3` => Trả về 6... cứ thế cho đến hết.

Hàm hệ thống trong PHP:

Ví dụ: dùng `array_reduce()` tính tổng thanh toán giỏ hàng.

```
$cart = [  
    ['name' => 'Iphone', 'price' => 1000],  
    ['name' => 'Case', 'price' => 50],  
    ['name' => 'Charger', 'price' => 30]  
];  
  
$total = array_reduce($cart, function($carry, $item) {  
    return $carry + $item['price'];  
}, 0);  
  
echo "Tổng tiền: $" . $total;
```

Tổng tiền: \$1080

Hàm hệ thống trong PHP:

Nhóm hàm cắt và tách mảng:

- **array_slice**(\$array, \$offset, \$length) – Cắt mảng \$array từ vị trí \$offset với độ dài \$length.
- **array_splice**(\$array, \$offset, \$length, \$replacement) – Cắt mảng \$array từ vị trí \$offset với độ dài \$length và thay thế bằng \$replacement.
- **array_chunk**(\$array, \$size) – Chia mảng (\$array thành nhiều mảng con với kích thước \$size.

```
$fruits = ["Táo", "Chuối", "Cam", "Xoài"];  
// Cắt từ vị trí 1, lấy 2 phần tử và thay bằng "Lựu"  
array_splice($fruits, 1, 2, ["Lựu"]);  
echo "<pre>";  
print_r($fruits);  
echo "</pre>";
```

```
Array  
(  
    [0] => Táo  
    [1] => Lựu  
    [2] => Xoài  
)
```

Hàm hệ thống trong PHP:

Nhóm hàm lấy thông tin mảng:

- `count()` – Đếm số phần tử.
- `array_keys()` – Lấy danh sách key.
- `array_values()` – Lấy danh sách giá trị.
- `array_unique()` – Loại bỏ giá trị trùng

Hàm hệ thống trong PHP:

Nhóm hàm toán học và tập hợp trong mảng:

- `array_sum()` – Tổng giá trị.
- `array_product()` – Tích giá trị.
- `array_intersect()` – Giao của các mảng (phần tử giống nhau).
- `array_diff($a, $b, $c)` – Phần tử chỉ có trong mảng \$a.

```
$a = [1, 2, 3, 4];  
$b = [3, 4, 5, 6];  
$result = array_intersect($a, $b);  
$result2 = array_diff($a, $b);  
echo "<pre>";  
print_r($result);  
print_r($result2);  
echo "</pre>";
```

```
Array  
(  
    [2] => 3  
    [3] => 4  
)  
Array  
(  
    [0] => 1  
    [1] => 2  
)
```

Hàm hệ thống trong PHP:

Các hàm toán học cơ bản:

- `abs()` – Giá trị tuyệt đối.
- `round()` – Làm tròn số.
- `floor()` – Làm tròn xuống.
- `ceil()` – Làm tròn lên.
- `sqrt()` – Căn bậc hai.
- `pow()` – Lũy thừa.
- `min()`, `max()` – Giá trị nhỏ nhất và lớn nhất.
- `rand(a,b)` – Lấy số ngẫu nhiên từ a tới b hoặc bất kỳ nếu không có tham số.
- `number_format()` – Định dạng số (thường dùng cho tiền).

Hàm hệ thống trong PHP:

Nhóm hàm thời gian và ngày tháng:

- `time()` – Timestamp hiện tại (số giây từ 1/1/1970).
- `date()` – Định dạng thời gian.

```
echo date('Y-m-d H:i:s'). '<br>';  
echo date('d/m/Y'). '<br>';  
echo date('H:i');
```

```
2026-02-15 14:34:57  
15/02/2026  
14:34
```

- `mktime()` – Tạo timestamp từ giờ/phút/giây/ngày/tháng/năm.

```
// mktime(hour, minute, second, month, day, year)  
$ts = mktime(0, 0, 0, 2, 15, 2026);  
echo $ts. '<br>';  
echo date('Y-m-d', $ts);
```

```
1771110000  
2026-02-15
```

Hàm hệ thống trong PHP:

Nhóm hàm thời gian và ngày tháng:

- Chuyển chuỗi sang thời gian: strtotime()

```
$ts = strtotime('2026-02-15 18:30:00');  
echo date('d/m/Y H:i', $ts). '<br>';  
echo date('Y-m-d', strtotime('+1 day')). '<br>';  
echo date('Y-m-d', strtotime('+1 week')). '<br>';  
echo date('Y-m-d', strtotime('last Monday')). '<br>';
```

```
15/02/2026 18:30  
2026-02-16  
2026-02-22  
2026-02-09
```

Xử lý thời gian với đối tượng DateTime():

Từ PHP 5.2 trở đi, đối tượng DateTime() ra đời theo phong cách lập trình hướng đối tượng (OOP), giúp việc xử lý thời gian trở nên cực kỳ chuyên nghiệp và chính xác (đặc biệt là khi xử lý múi giờ).

- Khởi tạo đối tượng DateTime:

```
// Lấy thời gian hiện tại
$now = new DateTime();

// Lấy một thời điểm cụ thể
$birthday = new DateTime('1995-12-25 08:30:00');

// Dùng "từ ngữ con người" (PHP rất thông minh!)
$tomorrow = new DateTime('tomorrow'); // ngày mai
$nextWeek = new DateTime('+1 week'); // tuần sau
```

Xử lý thời gian với đối tượng DateTime():

- Định dạng hiển thị format:

Dữ liệu trong đối tượng DateTime cần được định dạng bằng hàm format() trước khi hiển thị bằng echo.

```
$date = new DateTime();  
echo $date->format('d/m/Y H:i:s');
```

15/02/2026 15:20:38

- d: Ngày
- m: Tháng
- Y: Năm
- H: Giờ
- i: Phút
- s: Giây

Xử lý thời gian với đối tượng DateTime():

- Chỉnh sửa thời gian với phương thức modify.

```
$date = new DateTime('2026-02-15');  
$date->modify('+2 days');  
$date->modify('+1 month');  
echo $date->format('Y-m-d');
```

2026-03-17

Lưu ý: Khi dùng modify thì \$date sẽ thay đổi sang ngày mới. Vì vậy nếu muốn giữ nguyên ngày gốc thì ta dùng DateTimeImmutable() thay cho DateTime().

```
$today = new DateTimeImmutable('2026-02-15');  
$tomorrow = $today->modify('+1 day');  
//Cần dùng biến mới để lưu lại giá trị sau khi đã thay đổi  
echo $today->format('Y-m-d') . "<br>";  
echo $tomorrow->format('Y-m-d');
```

2026-02-15
2026-02-16

Xử lý thời gian với đối tượng DateTime():

- Tính toán khoảng cách giữa hai ngày với hàm diff().

```
$today = new DateTime('2026-02-15');  
$tet    = new DateTime('2027-02-17'); // Mừng 1 Tết năm sau  
  
$interval = $today->diff($tet);  
  
echo "Còn " . $interval->format('%a') . " ngày nữa là đến Tết!."<br>;  
echo "Chi tiết: " . $interval->format('%y năm, %m tháng, %d ngày');
```

Còn 367 ngày nữa là đến Tết!
Chi tiết: 1 năm, 0 tháng, 2 ngày

Xử lý thời gian với đối tượng DateTime():

- Làm việc với múi giờ kết hợp cùng DateTimeZone.

```
$tz = new DateTimeZone('Asia/Ho_Chi_Minh');  
$date = new DateTime('now', $tz);  
  
echo "Giờ Việt Nam: " . $date->format('H:i')." <br>";  
  
// Chuyển sang giờ New York  
$tz_new = new DateTimeZone('America/New_York');  
$date->setTimezone($tz_new );  
echo "Giờ New York: " . $date->format('H:i');
```

```
Giờ Việt Nam: 21:40  
Giờ New York: 09:40
```

Liên kết file trong PHP:

- **include (\$path);**: Nếu không tìm thấy file, PHP sẽ đưa ra một Cảnh báo (Warning), nhưng vẫn tiếp tục chạy các mã bên dưới.
- **require (\$path);**: Nếu không tìm thấy file, PHP sẽ đưa ra một Lỗi nghiêm trọng (Fatal Error) và dừng toàn bộ chương trình ngay lập tức.
- Dùng **include_once(\$path)** hoặc **require_once(\$path)**: PHP sẽ kiểm tra xem file đó đã được liên kết trước đó chưa. Nếu có rồi, nó sẽ không kéo vào nữa.

Có 2 loại đường dẫn:

- Đường dẫn tương đối: `include 'libs/functions.php';` (Tính từ vị trí file hiện tại).
- Đường dẫn tuyệt đối (An toàn nhất): Dùng hằng số `__DIR__` để lấy thư mục hiện tại của file. `require __DIR__ . '/config.php';`

Liên kết file trong PHP:

Ví dụ: Giả sử có 2 file là header.php và footer.php cùng thư mục với file index.php. Nội dung file index.php có thể kéo 2 file kia vào như sau:

```
<?php
// Kéo header vào
require 'header.php';
?>

<main>
    <p>Đây là nội dung chính của trang chủ.</p>
</main>

<?php
// Kéo footer vào
include 'footer.php';
?>
```

CÁC ĐỐI TƯỢNG QUAN TRỌNG TRONG PHP

Nội dung chính

PHP Regex

PHP File

PHP Form

Session & Cookie

Regex (Regular Expressions) hay còn gọi là Biểu thức chính quy là một chuỗi các ký tự tạo thành một mẫu tìm kiếm (search pattern).

Ví dụ: Thay vì tìm chữ "admin", ta có thể dùng Regex để tìm: "một chuỗi bắt đầu bằng chữ 'a', kết thúc bằng số, và có độ dài từ 5-10 ký tự".

Trong PHP, Regex luôn nằm giữa hai dấu gạch chéo / /.

Ví dụ: /php/ -> Tìm chữ "php" trong đoạn văn.

Chú ý: Để học và kiểm tra thử mẫu Regex của mình có hoạt động chuẩn hay không, bạn có thể kiểm tra chúng trên trang <https://regex101.com/>

Các ký hiệu phổ biến sử dụng trong Regex:

Ký hiệu	Ý nghĩa	Ví dụ
^	Bắt đầu chuỗi	/^Hello/ (Chuỗi phải bắt đầu bằng Hello)
\$	Kết thúc chuỗi	/world\$/ (Chuỗi phải kết thúc bằng world)
[abc]	Tìm bất kỳ ký tự nào trong ngoặc	/[0-9]/ (Tìm một chữ số bất kỳ)
+	Có 1 hoặc nhiều ký tự	/a+/ (Tìm ít nhất một chữ 'a' hoặc nhiều hơn)
*	Có 0 hoặc nhiều ký tự	/a*/ (Có 'a' cũng được, không có cũng không sao)
?	Có 0 hoặc 1 ký tự (Tùy chọn)	/https?/ (Khớp cả http và https)
.	Đại diện cho bất kỳ ký tự nào	/a.c/ (Khớp abc, axc, a1c...)
{n}	Lặp n lần	/0{3}/ (Tìm 3 số 0 liên tục 000)
{n,}	Lặp ít nhất n lần	/0{3,}/ (Tìm ít nhất 3 số 0 liên tục 000)
{n,m}	Lặp từ n tới m lần	/a{1,2}/ (Tìm a hoặc aa)

Các ký tự đặc tả dùng trong Regex:

Mẫu	Ý nghĩa
\d	Chữ số từ 0 đến 9
\D	Không phải số
\w	Ký tự chữ, số hoặc gạch dưới
\W	Không phải là chữ, số hoặc gạch dưới
\s	Khoảng trắng
\S	Không phải khoảng trắng

Các hàm Regex phổ biến trong PHP:

`preg_match()` Kiểm tra xem có khớp không:

Hàm này trả về 1 nếu tìm thấy, 0 nếu không thấy. Thường dùng để kiểm tra định dạng (Validate).

```
$string = "Học lập trình PHP tại UTH";  
$pattern = "/PHP/";  
  
if (preg_match($pattern, $string)) {  
    echo "Tìm thấy PHP trong chuỗi!";  
}
```

Tìm thấy PHP trong chuỗi!

Các hàm Regex phổ biến trong PHP:

preg_replace() Tìm và thay thế:

Hàm này cực mạnh để dọn dẹp dữ liệu hoặc ẩn thông tin nhạy cảm.

```
$string = "Số điện thoại của tôi là 090-123-456";  
$pattern = "/[0-9]/"; // Tìm tất cả các chữ số  
  
// Thay tất cả số bằng dấu *  
echo preg_replace($pattern, "*", $string);
```

Số điện thoại của tôi là ***_***_***

Các hàm Regex phổ biến trong PHP:

preg_match_all() Lấy ra tất cả các kết quả:

Dùng khi ta muốn trích xuất (extract) tất cả thông tin khớp mẫu.

```
$html = "<b>PHP</b> và <b>MySQL</b>";  
$pattern = "/<b>(.*?)</b>/"; // Tìm nội dung bên trong thẻ b  
  
preg_match_all($pattern, $html, $matches);  
echo "<pre>";  
print_r($matches[1]);
```

```
Array  
(  
    [0] => PHP  
    [1] => MySQL  
)
```

Một số mẫu Regex thông dụng:

- Kiểm tra email:

`/^[\\w.-]+@[\\w.-]+\\. [a-zA-Z]{2,6}$ /`

- Kiểm tra số điện thoại Việt Nam:

`/^(03|05|07|08|09)[0-9]{8}$ /`

- Kiểm tra mật khẩu ít nhất 8 ký tự có chữ hoa, thường và số:

`/^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d).{8,}$ /`

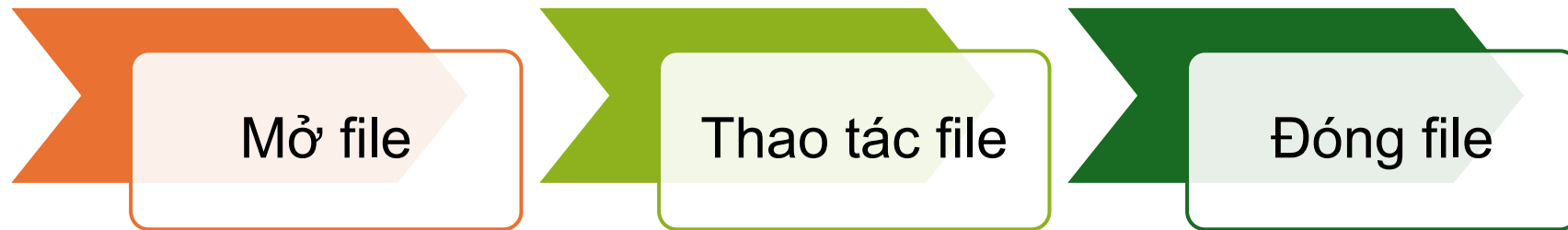
- Lấy URL trong chuỗi:

`/https?:\\V[^\\s]+ /`

PHP hỗ trợ thao tác file mạnh mẽ. Các thao tác chính:

- Tạo file
- Đọc file
- Ghi file
- Xóa file
- Upload file

Quy trình: Mở → Thao tác → Đóng



Hàm mở file `fopen()`:

Cú pháp: `fopen(filename, mode)`

Các mode phổ biến:

- `r` – đọc
- `w` – ghi, xóa nội dung cũ
- `a` – ghi nối
- `x` – tạo file mới
- `r+` đọc và ghi (file phải tồn tại)
- `w+` đọc và ghi (xóa nội dung cũ)
- `a+` – đọc và ghi (nối vào cuối file)

Ví dụ:

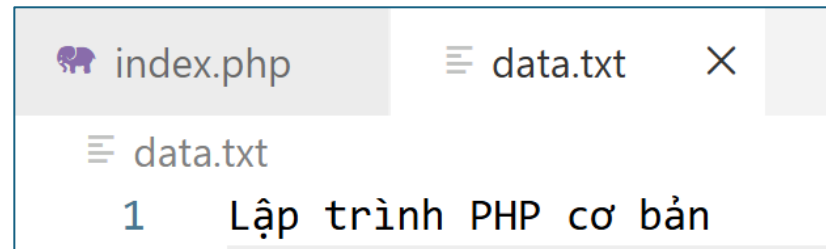
```
$file = fopen("data.txt", "w");
```

Xử lý lỗi bằng toán tử `@` và `die()`

```
$file = @fopen("data.txt", "r")  
or die("Không thể mở file");
```

Hàm ghi file fwrite() và đóng file fclose():

```
$file = fopen("data.txt", "w");  
fwrite($file, "Lập trình PHP cơ bản");  
fclose($file);
```



Đọc toàn bộ nội dung file với hàm file_get_contents():

```
$content = file_get_contents("data.txt");  
echo $content;
```

Lập trình PHP cơ bản

Đọc từng dòng nội dung file với hàm fgets():

```
$file = fopen("data.txt", "r");  
while(!feof($file)) {  
    //feof: Kiểm tra đã đến cuối file chưa?  
    echo fgets($file) . "<br>";  
}  
fclose($file);
```

Đọc từng ký tự trong file với hàm fgetc():

```
$file = fopen("data.txt", "r");  
while(!feof($file)) {  
    echo fgetc($file);  
}  
fclose($file);
```

Ghi nội dung vào file bằng file_put_contents():

```
$data = "Chào mừng bạn đến với PHP 2026!";  
file_put_contents('log.txt', $data); // Ghi đè  
// Nếu muốn ghi thêm vào cuối file (append):  
file_put_contents('log.txt', $data, FILE_APPEND);
```

Các hàm hệ thống khác:

- file_exists(\$filename): Kiểm tra file tồn tại.
- filesize(\$filename): Trả về kích thước file (đơn vị byte).
- unlink(\$filename): Xóa một file vĩnh viễn.
- rename(\$old, \$new): Đổi tên hoặc di chuyển file.
- copy(\$source, \$dest): Sao chép file.
- pathinfo(\$filename): Trả về mảng chứa thông tin file (đuôi file, tên thư mục...).

Làm việc với path và folder:

- `mkdir($path, $mode, $recursive)`: Tạo thư mục mới.
 - `$mode`: Quyền truy cập (mặc định là 0777).
 - `$recursive`: Nếu để `true`, nó sẽ tạo luôn cả các thư mục cha nếu chúng chưa tồn tại.
- `rmdir($path)`: Xóa thư mục rỗng.
- `is_dir($path)`: Kiểm tra đường dẫn có phải thư mục không.
- `getcwd()`: Lấy đường dẫn thư mục hiện tại đang làm việc.
- `chdir($path)`: Thay đổi thư mục làm việc hiện tại sang một chỗ khác.
- `scandir()`: Trả về mảng chứa tất cả file và thư mục con.

Làm việc với path và folder:

- glob(): Lọc file theo mẫu giống Regex đơn giản.

```
// Chỉ lấy các file có đuôi .jpg trong thư mục uploads
$images = glob("uploads/*.jpg");

foreach ($images as $img) {
    echo "Ảnh: $img <br>";
}
```

- pathinfo(): bóc tách đường dẫn thành các phần nhỏ.

Ghi chú: trong PHP dấu chấm “.” đại diện cho thư mục hiện tại, còn hai dấu chấm “..” đại diện cho thư mục cha.

Phương thức GET

Dữ liệu gửi qua GET sẽ hiển thị trực tiếp trên thanh địa chỉ (URL) dưới dạng các tham số.

- **Khi nào dùng:** Tìm kiếm, lọc sản phẩm, phân trang (những dữ liệu không cần bảo mật và có thể lưu Bookmark).
- **Đặc điểm:** Giới hạn dung lượng chuỗi (khoảng 2000 ký tự).

Mã HTML

```
<form action="search.php" method="GET">  
    Tìm kiếm: <input type="text" name="query">  
    <button type="submit">Tìm</button>  
</form>
```

Mã PHP

```
// search.php  
$keyword = $_GET['query'] ?? 'Trống';  
echo "Bạn đang tìm kiếm với từ khóa: ";  
echo htmlspecialchars($keyword);
```

← ↻ 🏠 ⓘ localhost/BASE_PHP/search.php?query=PHP

Bạn đang tìm kiếm với từ khóa: PHP

Phương thức POST

Dữ liệu gửi qua POST sẽ được ẩn trong thân của yêu cầu HTTP (HTTP Body).

- **Khi nào dùng:** Đăng ký, đăng nhập, gửi bài viết, thanh toán (những dữ liệu nhạy cảm hoặc dung lượng lớn).
- **Đặc điểm:** Bảo mật hơn GET, không giới hạn dung lượng (phụ thuộc vào cấu hình Server).

Mã HTML

```
<form action="register.php" method="POST">
    Username: <input type="text" name="user">
    Password: <input type="password" name="pass">
    <button type="submit">Đăng ký</button>
</form>
```

Mã PHP file register.php

```
$user = $_POST['user'];
$pass = $_POST['pass'];
echo "Chào mừng $user";
echo ", mật khẩu của bạn đã được tiếp nhận.";
```



Xử lý upload file trong PHP

Khi upload, dữ liệu không nằm trong \$_POST mà nằm trong biến siêu toàn cục \$_FILES.

Mã HTML

```
<form action="upload.php" method="POST" enctype="multipart/form-data">  
    Chọn ảnh đại diện: <input type="file" name="avatar">  
    <button type="submit">Tải lên</button>  
</form>
```

Mã PHP file upload.php

```
if ($_SERVER['REQUEST_METHOD'] == 'POST' && isset($_FILES['avatar'])) {  
    $file = $_FILES['avatar'];  
    $fileName = $file['name'];          // Tên file gốc  
    $fileTmp  = $file['tmp_name'];      // Đường dẫn tạm thời trên Server  
    $fileSize = $file['size'];          // Dung lượng file  
    $fileError = $file['error'];        // Mã lỗi (nếu có)  
    // Thư mục lưu trữ  
    $targetDir = "uploads/";  
    $targetFile = $targetDir . basename($fileName); // lấy tên file gốc  
    // Kiểm tra và di chuyển file từ thư mục tạm sang thư mục đích  
    if ($fileError === 0) {  
        if (move_uploaded_file($fileTmp, $targetFile)) {  
            echo "Upload thành công! File lưu tại: " . $targetFile;  
        } else {  
            echo "Có lỗi xảy ra khi di chuyển file.";  
        }  
    }  
}
```

Xử lý upload file trong PHP

Upload nhiều file cùng lúc.

Mã HTML

```
<input type="file" name="files[]" multiple>
```

Mã PHP

```
foreach ($_FILES["files"]["tmp_name"] as $key => $tmp) {  
    $name = $_FILES["files"]["name"][$key];  
    move_uploaded_file($tmp, "uploads/" . $name);  
}
```

Cookie

Cookie là những tệp văn bản nhỏ được Server gửi xuống và lưu trữ tại trình duyệt của người dùng. Mỗi khi người dùng truy cập lại Web đó, trình duyệt sẽ tự động gửi Cookie này lên lại Server.

- Đặc điểm: Dữ liệu nằm ở máy người dùng nên có thể bị xem hoặc sửa đổi (kém an toàn).
- Thời hạn: Có thể tồn tại rất lâu (vài tháng, vài năm) ngay cả khi tắt máy tính.

Ví dụ ứng dụng:

- Ghi nhớ tên đăng nhập.
- Ghi nhớ ngôn ngữ.
- Ghi nhớ theme (dark/light).
- Theo dõi hành vi người dùng.

Cookie

Cú pháp tạo cookie:

```
setcookie(name, value, expire, path, domain, secure, httponly);
```

- name (Bắt buộc): Tên của Cookie (ví dụ: "user_login").
- value: Giá trị muốn lưu.
- expires: Thời điểm hết hạn (tính bằng giây) sau khi đóng trình duyệt.
- path: Nếu để trống, cookie chỉ có tác dụng ở thư mục hiện tại. Hãy luôn để là "/" để nó có tác dụng toàn trang web.
- domain: Tên miền của cookie.
- secure: Nếu là true, cookie chỉ được gửi qua kết nối bảo mật HTTPS.
- httponly: Nếu là true, cookie chỉ truy cập được thông qua giao thức http.

Cookie

Ví dụ: Cách dùng cookie trong PHP:

```
// 1. Thiết lập Cookie (Tên, Giá trị, Thời gian hết hạn giây)
// Ở đây thầy đặt cookie tên 'user_lang' giá trị 'vi', hết hạn sau 1 ngày
setcookie("user_lang", "vi", time() + (86400 * 1), "/");
// 2. Truy cập Cookie thông qua biến hệ thống $_COOKIE
echo "Ngôn ngữ yêu thích của bạn là: " . ($_COOKIE['user_lang'] ?? 'Chưa chọn');
// 3. Xóa Cookie (Đặt thời gian hết hạn về quá khứ)
setcookie("user_lang", "", time() - 3600, "/");
```

Lưu ý:

- Luôn đặt setcookie() ở dòng đầu tiên của file, trước cả thẻ <html>.
- Để kiểm tra cookie trên trình duyệt: Nhấn F12 (hoặc chuột phải -> Inspect). Chọn tab Application. Ở cột bên trái, tìm mục Storage -> Cookies -> Chọn địa chỉ web muốn kiểm tra.

Session

Session (Phiên làm việc) lưu trữ dữ liệu trên Server. Server sẽ cấp cho mỗi người dùng một mã ID duy nhất (thường lưu mã này vào Cookie tên là PHPSESSID ở máy khách) để nhận diện.

- Đặc điểm: Bảo mật cao vì dữ liệu không nằm ở máy người dùng.
- Thời hạn: Thường kết thúc ngay khi em đóng trình duyệt.

Ví dụ ứng dụng:

- Lưu thông tin đăng nhập
- Lưu giỏ hàng
- Lưu quyền truy cập (role)
- Lưu dữ liệu tạm trong quá trình xử lý

Session

Cách dùng Session trng PHP:

```
// 1. Luôn phải gọi hàm này ở ĐẦU FILE (trước mọi thẻ HTML)
session_start();

// 2. Lưu dữ liệu vào Session
$_SESSION['username'] = "an_nguyen";
$_SESSION['role'] = "admin";

// 3. Truy cập Session ở bất kỳ trang nào khác
echo "Chào mừng quản trị viên: " . $_SESSION['username'];

// 4. Xóa Session
unset($_SESSION['username']); // Xóa 1 biến cụ thể
session_destroy();           // Hủy toàn bộ Session
```

Sự phối hợp giữa Session và Cookie.

Thực tế, Session vẫn cần đến Cookie.

- Khi bạn gọi `session_start()`, PHP tự tạo một mã ID ngẫu nhiên (ví dụ: abc123xyz).
- Mã ID này được gửi xuống trình duyệt và lưu vào một Cookie tên là PHPSESSID.
- Lần sau bạn truy cập, trình duyệt gửi mã abc123xyz lên, Server nhìn vào mã đó để xác định ID của bạn.

Lưu ý bảo mật:

- Cookie: Tuyệt đối không lưu mật khẩu hay thông tin thẻ tín dụng vào Cookie.
- Session Hijacking: Để bảo mật hơn, hãy gọi hàm `session_regenerate_id(true)` mỗi khi người dùng thay đổi trạng thái (như vừa đăng nhập thành công) để đổi mã ID mới, tránh bị hacker "cướp" phiên làm việc.

Ví dụ thực tế trong trang Login.

```
session_start();

// Giả sử kiểm tra username/password trong DB thành công
if ($login_success) {
    // 1. Đổi ngay ID mới trước khi lưu dữ liệu quan trọng
    session_regenerate_id(true);

    // 2. Lưu thông tin đăng nhập
    $_SESSION['user_id'] = 123;
    $_SESSION['is_logged_in'] = true;

    echo "Đăng nhập thành công với Session ID mới!";
}
```

LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG TRONG PHP

NỘI DUNG CHÍNH

Class & Object

Thuộc tính và phương thức

Hàm xây dựng và hàm hủy

Kế thừa và kỹ thuật nạp chồng

Phạm vi và sự ảnh hưởng

Abstract Class

Interface

Lớp (Class) và Đối tượng (Object)

- Class (Lớp): Là một bản thiết kế, một khuôn mẫu. Nó định nghĩa các đặc điểm và hành vi mà một đối tượng sẽ có.
- Object (Đối tượng): Là một thực thể cụ thể được tạo ra từ Class.

```
class Car {  
    // Nội dung bên trong class  
}  
  
$myCar = new Car(); // Khởi tạo một đối tượng
```

Thuộc tính (Properties) và Phương thức (Methods)

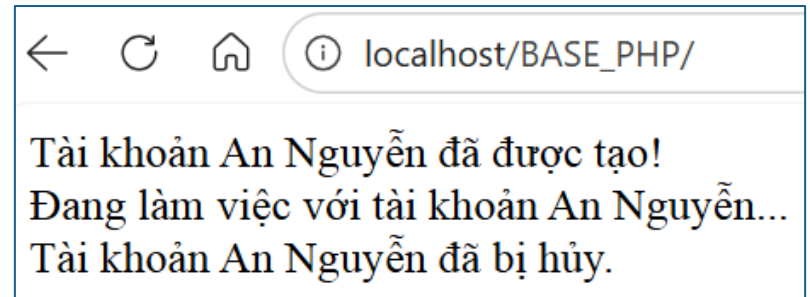
- Thuộc tính: Là các biến bên trong Class (đặc điểm của đối tượng).
- Phương thức: Là các hàm bên trong Class (hành động của đối tượng).

```
class MobilePhone {  
    public $brand; // Thuộc tính  
    public $color;  
  
    public function call() { // Phương thức  
        return "Đang gọi điện...";  
    }  
}  
  
$iphone = new MobilePhone();  
$iphone->brand = "Apple"; // Gán giá trị cho thuộc tính  
echo $iphone->call();      // Gọi phương thức
```

Hàm xây dựng (__construct) và Hàm hủy (__destruct)

- __construct(): Tự động chạy ngay khi đối tượng được khởi tạo (new).
- __destruct(): Tự động chạy khi đối tượng bị hủy hoặc script kết thúc.

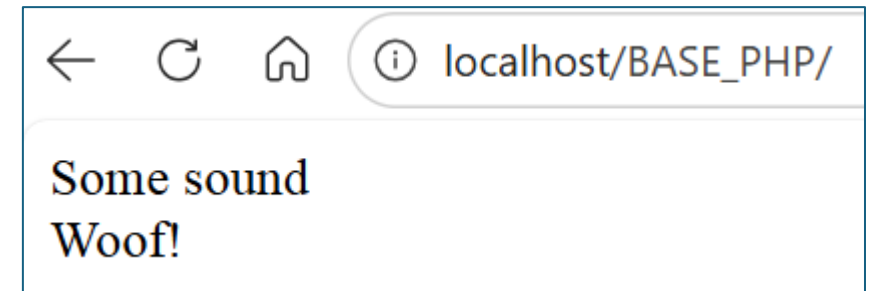
```
class User {  
    public $name;  
  
    public function __construct($n) {  
        $this->name = $n;  
        echo "Tài khoản {$this->name} đã được tạo!<br>";  
    }  
    public function __destruct() {  
        echo "Tài khoản {$this->name} đã bị hủy.";  
    }  
}  
$user1 = new User("An Nguyễn");  
echo "Đang làm việc với tài khoản {$user1->name}...<br>";
```



Kế thừa (Inheritance)

- Cho phép Class con sử dụng lại các thuộc tính và phương thức của Class cha.
- Dùng từ khóa extends.

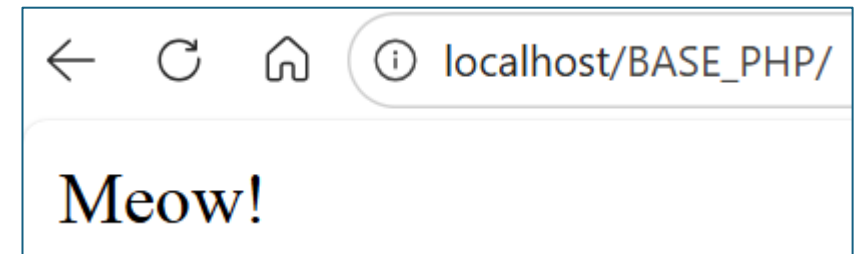
```
class Animal {  
    public function sound() {  
        echo "Some sound";  
    }  
}  
  
class Dog extends Animal {  
    public function bark() {  
        echo "Woof!";  
    }  
}  
  
$dog = new Dog();  
echo $dog->sound(). "<br>"; // từ class cha  
echo $dog->bark(); // từ class con
```



Ghi đè phương thức (Overriding)

Class con định nghĩa lại phương thức của class cha.

```
class Animal {  
    public function sound() {  
        echo "Some sound";  
    }  
}  
  
class Cat extends Animal {  
    public function sound() {  
        echo "Meow!";  
    }  
}  
  
$cat = new Cat();  
$cat->sound();  
?>
```



Kỹ thuật nạp chồng (Overloading)

PHP không hỗ trợ overloading như Java/C#, nhưng hỗ trợ overloading động qua các Magic Methods để xử lý các thuộc tính hoặc phương thức chưa được khai báo hoặc không được phép truy cập. Có 2 loại:

- Property Overloading: Dùng `__get()`, `__set()`, `__isset()`, `__unset()`.
- Method Overloading: Dùng `__call()` (cho phương thức thường) và `__callStatic()` (cho phương thức static).

Thay vì tạo nhiều hàm, ta dùng `__call()` để hứng tất cả các cuộc gọi đến những hàm không tồn tại và xử lý logic tùy biến.

Kỹ thuật nạp chồng (Overloading)

```
class SmartCalculator {  
    public function __call($name, $arguments) {  
        if ($name == 'add') {  
            if (count($arguments) == 2) {  
                return $arguments[0] + $arguments[1];  
            }  
            if (count($arguments) == 3) {  
                return $arguments[0] + $arguments[1] + $arguments[2];  
            }  
        }  
    }  
}  
  
$cal = new SmartCalculator();  
echo $cal->add(1, 2);           // Kết quả: 3  
echo $cal->add(1, 2, 3);        // Kết quả: 6
```

Giải thích: Hàm add thực tế không tồn tại trong Class, nhưng nhờ __call (Overloading trong PHP), chúng ta vẫn có thể gọi nó với số lượng tham số khác nhau.

Phạm vi truy cập và sự ảnh hưởng

Modifier	Tầm ảnh hưởng
public	Có thể truy cập từ bất cứ đâu (mặc định).
protected	Chỉ có thể truy cập bên trong Class đó và các Class kế thừa nó.
private	Chỉ có thể truy cập duy nhất bên trong Class định nghĩa nó.

```
class Person {  
    public $name = "An";  
    private $age = 20;  
    protected $address = "HCM";  
  
    public function showInfo() {  
        echo $this->name;  
        echo $this->age;  
        echo $this->address;  
    }  
}  
class Child extends Person {  
    public function showAddress() {  
        echo $this->address; // OK  
        // echo $this->age; // ERROR  
    }  
}
```

Lớp trừu tượng (Abstract Class)

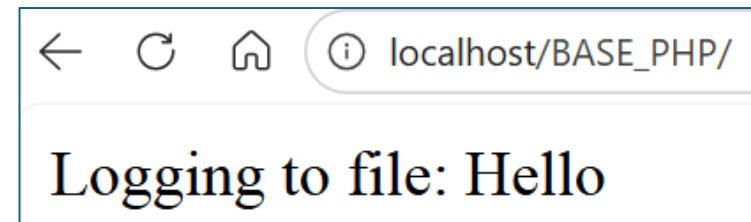
Abstract class đóng vai trò như một bản mẫu dở dang. Ta không thể khởi tạo đối tượng trực tiếp từ nó. Các Class con bắt buộc phải định nghĩa lại các phương thức trừu tượng.

```
abstract class Shape {  
    abstract public function calculateArea(); // Phương thức không có nội dung  
}  
  
class Square extends Shape {  
    public $side = 4;  
    public function calculateArea() {  
        return $this->side * $this->side;  
    }  
}
```

Interface

Interface chỉ chứa khai báo các phương thức mà một Class phải thực hiện, nhưng không quan tâm Class đó thực hiện như thế nào. Một Class có thể thực thi (implements) nhiều Interface cùng lúc.

```
interface Logger {  
    public function log($msg);  
}  
  
class FileLogger implements Logger {  
    public function log($msg) {  
        echo "Logging to file: $msg";  
    }  
}  
  
$log = new FileLogger();  
$log->log("Hello");
```



So sánh nhanh Abstract Class và Interface

Đặc điểm	Abstract Class	Interface
Khởi tạo	Không thể	Không thể
Thuộc tính	Có thể có biến	Chỉ có hằng số (const)
Phương thức	Có thể có code hoặc không	Chỉ có tên hàm (rỗng)
Kế thừa	Đơn kế thừa (extends 1 cha)	Đa thực thi (implements nhiều)

Phần 2. MySQL

MySQL là một hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) mã nguồn mở, phổ biến nhất thế giới, được phát triển và hỗ trợ bởi Oracle Corporation. Nó sử dụng ngôn ngữ SQL để quản lý dữ liệu trong các bảng, hoạt động theo mô hình Client-Server, nổi tiếng với tốc độ, tính ổn định và khả năng mở rộng. MySQL rất thân thiện với các ngôn ngữ lập trình phổ biến, đặc biệt là PHP, giúp việc xây dựng các trang web thương mại điện tử, ứng dụng dữ liệu lớn trở nên đơn giản, nhanh chóng và tiết kiệm chi phí.

NỘI DUNG CHÍNH

MySQL và
phpMyAdmin

Ngôn ngữ truy vấn
SQL

Kết nối PHP với
MySQL

MySQL là gì?

- MySQL là hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) phổ biến nhất thế giới.
- Dùng mô hình bảng (table) để lưu trữ dữ liệu.
- Miễn phí, mạnh mẽ, dễ dùng, thường đi kèm với PHP trong mô hình LAMP/XAMPP.

phpMyAdmin là công cụ web giúp ta thao tác với MySQL bằng giao diện đồ họa (nhấn chuột) thay vì chỉ gõ lệnh khô khan.

Cách mở: Trên XAMPP, nhấn nút Start --> Admin dòng MySQL hoặc truy cập <http://localhost/phpmyadmin>.

Thao tác chính: Tạo Database, Tạo Table, Thêm/Sửa/Xóa dữ liệu thủ công.

Giao diện trang chính phpMyAdmin

The screenshot displays the phpMyAdmin web interface in a browser window. The address bar shows 'localhost/phpmyadmin/'. The interface is in Vietnamese and features a top navigation bar with tabs: 'Cơ sở dữ liệu' (Databases), 'SQL', 'Tình trạng' (Status), 'Các tài khoản người dùng' (Users), 'Xuất' (Export), 'Nhập' (Import), 'Cài đặt' (Settings), 'Bản sao' (Backup), 'Biến' (Variables), and 'Thêm' (Add). The left sidebar contains a tree view of databases: 'information_schema', 'mysql', 'performance_schema', 'phpmyadmin', and 'test'. The main content area is divided into three panels:

- Cài đặt cơ bản** (Basic Settings): Shows 'Server connection collation' set to 'utf8mb4_unicode_ci' and a link to 'Cài đặt thêm' (Additional Settings).
- Cài đặt diện mạo** (Appearance Settings): Shows 'Ngôn ngữ (Language)' set to 'Tiếng Việt - Vietnamese' and 'Chủ đề' (Theme) set to 'pmahomme'.
- Máy chủ cơ sở dữ liệu** (Database Server Information): Lists system details such as 'Máy chủ: 127.0.0.1 via TCP/IP', 'Kiểu máy chủ: MariaDB', 'Server connection: SSL is not being used', 'Phiên bản máy phục vụ: 10.4.32-MariaDB', 'Phiên bản giao thức: 10', 'Người dùng: root@localhost', and 'Bảng mã máy chủ: UTF-8 Unicode (utf8mb4)'.
- Máy chủ Web** (Web Server Information): Lists web server details such as 'Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.0.30', 'Phiên bản cơ sở dữ liệu máy khách: libmysql - mysqlnd 8.0.30', 'Phần mở rộng PHP: mysqli, curl, mbstring', and 'Phiên bản PHP: 8.0.30'.

Thao tác cơ bản trong phpMyAdmin

Tạo database:

- Mở phpMyAdmin.
- Chọn tab Databases.
- Nhập tên database → Create (nên chọn bảng mã utf8_general_ci để có hỗ trợ tiếng Việt).

Tạo bảng:

- Chọn database.
- Nhập tên bảng + số cột → Go.
- Khai báo: Tên cột, Kiểu dữ liệu (INT, VARCHAR...), Khóa chính (PRIMARY KEY), Tự tăng (AUTO_INCREMENT),...

Thao tác cơ bản trong phpMyAdmin

Tạo dòng dữ liệu cho bảng:

- Chọn bảng.
- Chọn tab insert (chèn).
- Nhập nội dung cho các cột, sau đó chọn thực hiện.

Các thao tác khác với bảng có thể thực hiện ở giao diện Duyệt bảng hoặc sử dụng tab SQL để nhập lệnh SQL.

Ngôn ngữ truy vấn SQL

Để giao tiếp với MySQL, chúng ta dùng ngôn ngữ SQL (Structured Query Language).

Tạo database:

```
CREATE DATABASE school;
```

Xóa database:

```
DROP DATABASE school;
```

Tạo bảng:

```
CREATE TABLE students (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100),  
  age INT,  
  email VARCHAR(100)  
);
```

Thêm cột:

```
ALTER TABLE students ADD address VARCHAR(255);
```

Xóa bảng:

```
DROP TABLE students;
```

Tạo bảng:

```
CREATE TABLE students (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100),  
    age INT,  
    email VARCHAR(100)  
);
```

Các thao tác với dữ liệu (CRUD):

CREATE (Thêm dữ liệu)

```
INSERT INTO users (username, email) VALUES ('an_nguyen', 'an@example.com');
```

READ (Đọc dữ liệu)

```
SELECT * FROM students; -Lấy toàn bộ  
SELECT * FROM students WHERE age > 18; -Lấy có điều kiện  
SELECT * FROM students ORDER BY age DESC; -Sắp xếp  
SELECT * FROM students LIMIT 5; -Giới hạn số dòng
```

UPDATE (Cập nhật dữ liệu)

```
UPDATE students SET age = 21 WHERE id = 1;
```

DELETE (Xóa dữ liệu)

```
DELETE FROM students WHERE id = 1;
```

INNER JOIN (Lấy phần chung)

Chỉ trả về những bản ghi mà cả hai bảng đều có dữ liệu khớp nhau.

Ví dụ: Lấy danh sách các đơn hàng và tên người mua tương ứng. (Nếu ai chưa mua hàng bao giờ, họ sẽ không hiện ra).

```
SELECT orders.id, users.username, orders.total_price  
FROM orders  
INNER JOIN users ON orders.user_id = users.id;
```

LEFT JOIN (Lấy hết bên trái)

Lấy tất cả dữ liệu từ bảng bên trái (bảng đứng trước chữ JOIN), và dữ liệu khớp từ bảng bên phải. Nếu bảng bên phải không có dữ liệu khớp, nó sẽ hiện NULL.

Ví dụ: Lấy tất cả khách hàng, ai có đơn hàng thì hiện đơn hàng đó, ai chưa mua thì hiện NULL.

```
SELECT users.username, orders.id  
FROM users  
LEFT JOIN orders ON users.id = orders.user_id;
```

GROUP BY NHÓM DỮ LIỆU & THỐNG KÊ

GROUP BY dùng để nhóm các hàng có cùng giá trị lại với nhau. Nó thường đi đôi với các Hàm tổng hợp (Aggregate Functions) như:

- COUNT(): Đếm số lượng.
- SUM(): Tính tổng.
- AVG(): Tính trung bình.
- MAX() / MIN(): Lớn nhất / Nhỏ nhất.

Ví dụ: Đếm số đơn hàng của mỗi người.

```
SELECT user_id, COUNT(id) AS total_orders  
FROM orders  
GROUP BY user_id;
```

GROUP BY NHÓM DỮ LIỆU & THỐNG KÊ

Dùng HAVING để lọc điều kiện sau khi nhóm.

Ví dụ: Tìm những khách hàng đã chi tiêu tổng cộng trên 1.000.000 VNĐ.

```
SELECT users.username, SUM(orders.total_price) AS total_spent  
FROM users  
JOIN orders ON users.id = orders.user_id  
GROUP BY users.id  
HAVING total_spent > 1000000;
```

Tổng kết thứ tự câu lệnh SQL

1. SELECT (Chọn cột)
2. FROM (Từ bảng nào)
3. JOIN (Kết nối bảng nào)
4. WHERE (Lọc dữ liệu thô)
5. GROUP BY (Nhóm lại)
6. HAVING (Lọc sau khi nhóm)
7. ORDER BY (Sắp xếp)
8. LIMIT (Giới hạn số lượng)

Lưu ý: Khi dùng GROUP BY, tất cả các cột bạn chọn ở lệnh SELECT (mà không nằm trong hàm tổng hợp như SUM, COUNT) thì **bắt buộc** phải có mặt trong lệnh GROUP BY. Nếu không, MySQL sẽ báo lỗi hoặc trả về kết quả không chính xác!

Kết nối PHP với MySQL

Có 2 cách phổ biến:

- **MySQLi** (hỗ trợ hướng thủ tục và hướng đối tượng)
- **PDO** (mạnh hơn, hỗ trợ nhiều loại DB)

MySQLi (MySQL Improved):

Đây là phiên bản nâng cấp của thư viện mysql cũ. Nó được thiết kế dành riêng cho cơ sở dữ liệu MySQL.

- Ưu điểm: Tốc độ nhanh hơn một chút, dễ học nếu bạn đã quen với cách viết hàm tuần tự.
- Cách dùng: Có thể dùng theo kiểu hướng thủ tục (procedural) hoặc hướng đối tượng (OOP).

Kết nối MySQL hướng thủ tục:

```
$conn = mysqli_connect("localhost", "root", "", "testdb");  
  
if (!$conn) {  
    die("Kết nối thất bại: " . mysqli_connect_error());  
}  
echo "Kết nối thành công!";
```

Kết nối MySQL hướng đối tượng (nên dùng vì dễ mở rộng và bảo trì):

```
$conn = new mysqli("localhost", "root", "", "testdb");  
  
if ($conn->connect_error) {  
    die("Kết nối thất bại: " . $conn->connect_error);  
}  
echo "Kết nối thành công!";
```

SELECT dữ liệu hướng thủ tục:

```
$conn = mysqli_connect("localhost", "root", "", "testdb");  
$sql = "SELECT * FROM users";  
$result = mysqli_query($conn, $sql);  
while ($row = mysqli_fetch_assoc($result)) {  
    echo $row['name'] . "<br>";  
}
```

SELECT dữ liệu hướng đối tượng:

```
$conn = new mysqli("localhost", "root", "", "testdb");  
$sql = "SELECT * FROM users";  
$result = $conn->query($sql);  
while ($row = $result->fetch_assoc()) {  
    echo $row['name'] . "<br>";  
}
```

INSERT:

```
$sql = "INSERT INTO users (name, email) VALUES ('An', 'an@example.com')";  
$conn->query($sql);
```

UPDATE:

```
$sql = "UPDATE users SET name='New Name' WHERE id=1";  
$conn->query($sql);
```

DELETE:

```
$sql = "DELETE FROM users WHERE id=1";  
$conn->query($sql);
```

Lỗi bảo mật nguy hiểm SQL Injection:

Ví dụ:

```
$sql = "SELECT * FROM users WHERE username = '$user_input'";
```

Hacker có thể nhập \$user_input là ' OR '1'='1. Khi đó câu lệnh SQL trở thành:

```
$sql = "SELECT * FROM users WHERE username = '' OR '1'='1'";
```

--> Hacker đăng nhập được mà không cần mật khẩu!

Prepared Statements Giải pháp an toàn chống SQL Injection:

Khi ta viết một câu lệnh SQL có chứa biến từ người dùng (ví dụ: \$id), thay vì nối chuỗi trực tiếp, chúng ta dùng dấu chấm hỏi ? làm vị trí đặt chỗ (placeholder).

```
$stmt = $conn->prepare("SELECT * FROM users WHERE id = ?");  
$stmt->bind_param("i", $id); // i = integer  
$stmt->execute();  
$result = $stmt->get_result();  
$user = $result->fetch_assoc();
```

Hàm bind_param có nhiệm vụ: "Ràng buộc" giá trị của các biến vào các dấu ? đó và quan trọng nhất là lọc sạch dữ liệu trước khi gửi tới Database.

Cú pháp cơ bản của bind_param:

```
$stmt->bind_param("kiểu_dữ_liệu", $bien1, $bien2, ...);
```

Tham số đầu tiên là một chuỗi các ký tự đại diện cho kiểu dữ liệu của các biến theo sau.

Có 4 loại chính:

Ký tự	Kiểu dữ liệu	Ý nghĩa
i	integer	Số nguyên (ví dụ: 1, 100, -5)
d	double	Số thực (ví dụ: 15.5, 3.14)
s	string	Chuỗi văn bản (ví dụ: "An Nguyen", "abc@gmail.com")
b	blob	Dữ liệu nhị phân (ví dụ: dữ liệu hình ảnh, file)

Ví dụ minh họa:

Giả sử bạn muốn tìm một học sinh theo **tên** (chuỗi) và **tuổi** (số nguyên):

```
$name = "An";  
$age = 18;  
  
// Bước 1: Chuẩn bị mẫu (Prepare) với 2 dấu hỏi  
$stmt = $conn->prepare("SELECT * FROM students WHERE name = ? AND age = ?");  
  
// Bước 2: Ràng buộc tham số (Bind)  
// "si" nghĩa là tham số 1 là String, tham số 2 là Integer  
$stmt->bind_param("si", $name, $age);  
  
// Bước 3: Thực thi (Execute)  
$stmt->execute();  
  
$result = $stmt->get_result();
```

Xây dựng lớp Database bằng MySQLi (chuẩn dự án):

```
class Database {
    private $host = "localhost";
    private $user = "root";
    private $pass = "";
    private $dbname = "testdb";
    public $conn;

    public function __construct() {
        $this->conn = new mysqli($this->host, $this->user, $this->pass, $this->dbname);

        if ($this->conn->connect_error) {
            die("Kết nối thất bại: " . $this->conn->connect_error);
        }
    }
}

//Cách dùng
$db = new Database();
$result = $db->conn->query("SELECT * FROM users");
```

PDO là một lớp trừu tượng giúp PHP kết nối với nhiều loại cơ sở dữ liệu khác nhau thông qua cùng một giao diện.

PDO hỗ trợ hơn 12 loại DB:

- MySQL
- PostgreSQL
- SQLite
- SQL Server
- Oracle
- IBM DB2
- Firebird
- ...

Điểm mạnh nhất của PDO là bạn chỉ cần đổi DSN là có thể chuyển sang DB khác mà không phải viết lại toàn bộ code.

Cú pháp kết nối cơ bản

```
$host = "localhost";
$dbname = "testdb";
$username = "root";
$password = "";
try {
    $pdo = new PDO(
        "mysql:host=$host;dbname=$dbname;charset=utf8",
        $username,
        $password
    );
    // Bật chế độ báo lỗi
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    echo "Kết nối thành công!";
} catch (PDOException $e) {
    echo "Lỗi kết nối: " . $e->getMessage();
}
```

Truy vấn dữ liệu với PDO

SELECT tất cả

```
$stmt = $pdo->query("SELECT * FROM users");  
$data = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

SELECT với điều kiện (Prepared Statement)

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE id = ?");  
$stmt->execute([1]);  
$user = $stmt->fetch(PDO::FETCH_ASSOC);
```

Các kiểu fetch phổ biến:

- `PDO::FETCH_ASSOC`: trả về một mảng với tên khóa (key) chính là tên cột trong Database
- `PDO::FETCH_OBJ`: trả về một đối tượng vô danh (stdClass). Các tên cột sẽ trở thành các thuộc tính của đối tượng đó.
- `PDO::FETCH_BOTH` (Mặc định): trả về một mảng chứa cả tên cột và số thứ tự cột (bắt đầu từ 0). Nhược điểm: Mảng trả về sẽ nặng gấp đôi vì chứa dữ liệu lặp lại --> không nên dùng kiểu này để tiết kiệm bộ nhớ.
- `PDO::FETCH_COLUMN`: Nếu bạn chỉ muốn lấy giá trị của một cột duy nhất thì kiểu này cực kỳ tiện lợi vì nó trả về trực tiếp giá trị đó thay vì một mảng.
- `PDO::FETCH_CLASS`: lấy dữ liệu từ Database và đổ thẳng vào các thuộc tính của một Class mà bạn đã định nghĩa sẵn.

Ví dụ về PDO::FETCH_CLASS

```
class User {  
    public $name;  
    public $email;  
    public function showProfile() {  
        return "Tên: " . $this->name;  
    }  
}  
  
$stmt = $pdo->query("SELECT name, email FROM users");  
$stmt->setFetchMode(PDO::FETCH_CLASS, 'User');  
$user = $stmt->fetch();  
  
echo $user->showProfile();
```

fetchAll() và fetch()

- fetch(): Lấy ra một dòng duy nhất. Dùng khi ta tìm kiếm theo ID hoặc muốn dùng vòng lặp while để duyệt từng dòng một (tiết kiệm RAM khi dữ liệu lớn).
- fetchAll(): Lấy toàn bộ danh sách kết quả và bỏ vào một mảng lớn. Tiện khi dữ liệu ít và ta muốn dùng foreach để in ra ngoài giao diện.

```
// Lấy toàn bộ danh sách mảng kết hợp
$users = $stmt->fetchAll(PDO::FETCH_ASSOC);

foreach ($users as $u) {
    echo $u['name'] . "<br>";
}
```

Thiết lập chế độ mặc định

Thay vì lúc nào cũng phải viết `PDO::FETCH_ASSOC` trong từng hàm fetch, ta có thể thiết lập nó ngay lúc khởi tạo kết nối để code ngắn gọn hơn:

```
$options = [  
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,  
    PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION  
];  
$pdo = new PDO($dsn, $user, $pass, $options);  
  
// Bây giờ chỉ cần fetch() là nó tự hiểu là mảng kết hợp  
$user = $pdo->query("SELECT * FROM users")->fetch();
```

Kỹ thuật nâng cao: Named Placeholders

Thay vì dùng dấu ? dễ nhầm lẫn về thứ tự, PDO cho phép dùng các nhãn có tên dạng :label. Điều này giúp code cực kỳ dễ đọc và bảo trì.

```
$data = [  
    'title' => 'Lập trình PHP',  
    'author' => 'BVT',  
    'view' => 100  
];  
  
$sql = "INSERT INTO posts (title, author, views) VALUES (:title, :author, :view)";  
$stmt = $pdo->prepare($sql);  
  
// Bind kèm theo định nghĩa kiểu dữ liệu để tối ưu  
$stmt->bindValue(':title', $data['title'], PDO::PARAM_STR);  
$stmt->bindValue(':view', $data['view'], PDO::PARAM_INT);  
$stmt->execute();
```

Các kiểu dữ liệu (Data Types) trong PDO:

- PDO::PARAM_STR: Kiểu chuỗi (Mặc định).
- PDO::PARAM_INT: Kiểu số nguyên.
- PDO::PARAM_BOOL: Kiểu Boolean (true/false).
- PDO::PARAM_NULL: Kiểu dữ liệu NULL.
- PDO::PARAM_LOB: Kiểu dữ liệu lớn (Large Object dùng cho file/ảnh).

Ghi chú:

- Dùng bindValue() – Truyền giá trị: lấy giá trị của biến ngay tại thời điểm gọi hàm và gắn vào câu lệnh.
- Dùng bindParam() – Truyền tham chiếu: liên kết biến với câu lệnh theo kiểu tham chiếu (reference).

Tạo lớp Database dùng PDO chuẩn MVC:

```
class Database {
    private $host = "localhost";
    private $dbname = "testdb";
    private $username = "root";
    private $password = "";
    public $pdo;
    public function __construct() {
        try {
            $this->pdo = new PDO(
                "mysql:host=$this->host;dbname=$this->dbname;charset=utf8",
                $this->username,
                $this->password
            );
            $this->pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
        } catch (PDOException $e) {
            die("Lỗi kết nối: " . $e->getMessage());
        }
    }
}

//cách dùng
$db = new Database();
$stmt = $db->pdo->query("SELECT * FROM users");
$data = $stmt->fetchAll(PDO::FETCH_ASSOC)
```

Phần 3. Mô hình MVC

NỘI DUNG CHÍNH

Khái niệm và tư tưởng của MVC

Cơ chế Routing

Xây dựng bộ khung (framework) mini

Client-Side Rendering (CSR)

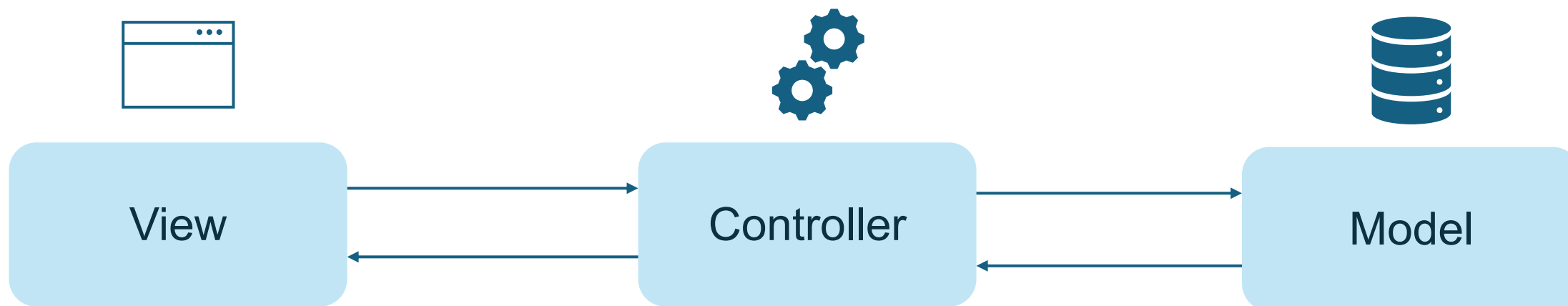
MVC (Model View Controller) là một mẫu kiến trúc phần mềm nhằm tách biệt các thành phần logic của một ứng dụng thành ba phần riêng biệt, giúp giảm thiểu sự phụ thuộc lẫn nhau:

- **M Model (Dữ liệu):** Model chịu trách nhiệm làm việc với Cơ sở dữ liệu. Nó chứa các hàm để lấy, thêm, sửa, xóa dữ liệu.
- **V View (Giao diện):** View là thành phần chịu trách nhiệm trình diễn dữ liệu, hiển thị dữ liệu và nhận tương tác từ người dùng. View nhận dữ liệu từ Controller và render ra giao diện cho người dùng cuối.
- **C Controller (Bộ điều khiển/điều hướng):** Điều phối Model và View. Tiếp nhận HTTP Request, xử lý tham số đầu vào, gọi Model thực hiện logic và chọn View phù hợp để phản hồi kết quả.

KHÁI NIỆM VÀ TƯ TƯỞNG CỦA MVC

Luồng hoạt động của MVC

- Router nhận URL → gọi Controller tương ứng.
- Controller gọi Model để lấy dữ liệu.
- Model truy vấn database → trả dữ liệu về Controller.
- Controller gửi dữ liệu sang View.
- View hiển thị HTML cho người dùng.



Lợi ích khi dùng MVC

- Dễ bảo trì: Nếu muốn đổi giao diện, ta chỉ cần sửa phần View, code xử lý dữ liệu ở Model vẫn giữ nguyên.
- Làm việc nhóm: Bạn chuyên Front-end chỉ làm việc trên các file View, bạn Back-end tập trung vào Model và Controller, không ai đụng chạm vào file của ai.
- Tái sử dụng: Một Model lấy dữ liệu bài viết có thể dùng chung cho cả View hiển thị trên Web và View trả về dữ liệu (JSON) cho App di động.

Nên dùng MVC khi nào

Tình huống	Có nên dùng MVC
Dự án lớn, nhiều tính năng	Rất nên
Team nhiều người	Nên
Cần bảo trì lâu dài	Nên
Web nhiều trang	Nên
Trang đơn SPA (React/Vue)	Không cần
Web nhỏ, đơn giản	Không nên

Trong các hệ thống MVC hiện đại, chúng ta không truy cập trực tiếp vào các file vật lý (như product.php). Thay vào đó, chúng ta sử dụng **Routing** để quyết định URL nào sẽ gọi hàm nào trong controller.

Cơ chế hoạt động:

- Người dùng gửi yêu cầu tới một URL (ví dụ: /product/detail/5).
- Router (Front Controller – thường là index.php) phân tích URL, tách URL thành các phần (product → controller, detail → action (method), 5 → parameter).
- Router tìm controller tương ứng (ProductController.php).
- Router gọi action tương ứng (detail(5)) và trả về kết quả (View hoặc JSON).

Routing trong MVC hoạt động dựa trên 3 thành phần chính:

- Route Table: Danh sách các route được định nghĩa.
- Router: Bộ phân tích URL và tìm route phù hợp.
- Dispatcher: Gọi đúng controller + action.

CƠ CHẾ ROUTING

Ví dụ: Routing đơn giản với cấu trúc file như sau:

```
| index.php  
| controllers/  
| _____ ProductController.php
```

ProductController.php

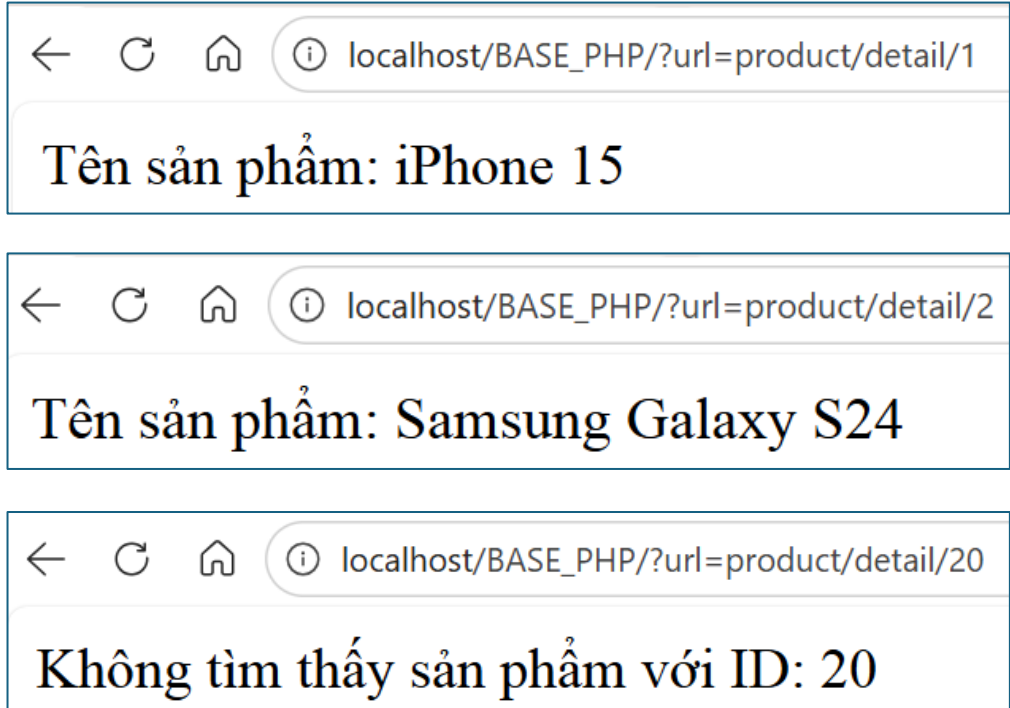
```
class ProductController {
    //Truy vấn DB lấy dữ liệu sản phẩm
    private $products = [
        ["id" => 1, "name" => "iPhone 15"],
        ["id" => 2, "name" => "Samsung Galaxy S24"],
        ["id" => 3, "name" => "Xiaomi 14"]
    ];

    public function detail($id) {
        foreach ($this->products as $product) {
            if ($product['id'] == $id) {
                echo "Tên sản phẩm: " . $product['name'];
                return;
            }
        }
        echo "Không tìm thấy sản phẩm với ID: " . $id;
    }
}
```

index.php

```
$url = $_GET['url'] ?? 'home/index';  
$url = explode('/', $url);  
  
$controllerName = ucfirst($url[0]) . 'Controller';  
$actionName = $url[1] ?? 'index';  
$param = $url[2] ?? null;  
  
require "controllers/".$controllerName.".php";  
  
$controller = new $controllerName();  
$controller->$actionName($param);
```

CÁC TÌNH HUỐNG DEMO



The image displays three browser window screenshots demonstrating the routing mechanism. Each window shows a browser address bar with a URL and a corresponding message below it.

- First Screenshot:** The address bar shows `localhost/BASE_PHP/?url=product/detail/1`. The message below is "Tên sản phẩm: iPhone 15".
- Second Screenshot:** The address bar shows `localhost/BASE_PHP/?url=product/detail/2`. The message below is "Tên sản phẩm: Samsung Galaxy S24".
- Third Screenshot:** The address bar shows `localhost/BASE_PHP/?url=product/detail/20`. The message below is "Không tìm thấy sản phẩm với ID: 20".

XÂY DỰNG BỘ KHUNG FRAMEWORK MINI

Thiết lập cấu trúc thư mục tiêu chuẩn

Một dự án chuyên nghiệp cần tách biệt phần mã nguồn logic và phần tài nguyên công khai (assets).

- Tạo thư mục dự án tên là `mvc_mini`.
- Bên trong, tạo các thư mục con sau:
 - `app/`: Chứa mã nguồn ứng dụng (Controller, Model, View).
 - `app/core/`: Chứa các lớp hạt nhân (App.php, Controller.php).
 - `public/`: Thư mục duy nhất mà người dùng có thể truy cập từ trình duyệt.

XÂY DỰNG BỘ KHUNG FRAMEWORK MINI

Cấu hình Front Controller và Rewrite URL

Mọi yêu cầu từ khách hàng phải được hội tụ về một điểm duy nhất là index.php để hệ thống bắt đầu xử lý.

Tạo file public/.htaccess file này chỉ thị cho Apache chuyển hướng mọi URL không tồn tại dưới dạng file vật lý về index.php

```
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^(.*)$ index.php?url=$1 [L,QSA]
```

Tạo file public/index.php đóng vai trò khởi tạo hệ thống

```
// Require các file cốt lõi
require_once '../app/core/App.php';
require_once '../app/core/Controller.php';
// Khởi tạo đối tượng điều hướng
$app = new App();
```

XÂY DỰNG BỘ KHUNG FRAMEWORK MINI

Xây dựng Lớp điều hướng chính (app/core/App.php) , phân tích URL để thực thi đúng phương thức (Action). Nội dung file App.php như sau:

```
class App {  
    // Các biến mặc định  
    protected $controller = "Home"; // Controller mặc định  
    protected $method = "index";    // Method mặc định  
    protected $params = [];          // Tham số mặc định  
  
    // public function __construct()  
  
    // Phương thức phân tích URL  
    private function parseUrl() {  
        // URL sẽ có dạng: /Controller/Method/Param1/Param2/...  
        if (isset($_GET['url'])) {  
            // Loại bỏ dấu gạch chéo ở đầu và cuối, sau đó phân tách thành mảng  
            return explode('/', filter_var(trim($_GET['url'], '/'), FILTER_SANITIZE_URL));  
        }  
    }  
}
```

XÂY DỰNG BỘ KHUNG FRAMEWORK MINI

```
public function __construct() {
    $url = $this->parseUrl();
    // 1. Kiểm tra sự tồn tại của Controller
    if (isset($url[0]) && file_exists('../app/controllers/'.ucfirst($url[0]).'.php')) {
        $this->controller = ucfirst($url[0]);
        unset($url[0]);
    }
    require_once '../app/controllers/' . $this->controller . '.php';
    $this->controller = new $this->controller;
    // 2. Kiểm tra sự tồn tại của Method trong Controller
    if (isset($url[1])) {
        if (method_exists($this->controller, $url[1])) {
            $this->method = $url[1];
            unset($url[1]);
        }
    }
    // 3. Xử lý tham số còn lại
    $this->params = $url ? array_values($url) : [];
    // 4. Thực thi Controller và Method với tham số truyền vào
    call_user_func_array([$this->controller, $this->method], $this->params);
}
```

XÂY DỰNG BỘ KHUNG FRAMEWORK MINI

Xây dựng Lớp cơ sở (app/core/Controller.php) , cung cấp các phương thức tiện ích cho tất cả các controller con. Nội dung file Controller.php như sau:

```
class Controller {  
    // Hàm gọi Model  
    public function model($model) {  
        require_once '../app/models/' . $model . '.php';  
        return new $model();  
    }  
  
    // Hàm gọi View kèm dữ liệu  
    public function view($view, $data = []) {  
        require_once '../app/views/' . $view . '.php';  
    }  
}
```

XÂY DỰNG BỘ KHUNG FRAMEWORK MINI

Controller mẫu mặc định (app/controllers/Home.php)

```
class Home extends Controller {  
  
    public function index($name = "Khách") {  
        // Chuẩn bị mảng dữ liệu để truyền sang View  
        $data = [  
            'name' => $name  
        ];  
  
        // Gọi phương thức view() từ lớp cha (Controller.php)  
        // Tham số 1: Tên file view (không cần đuôi .php)  
        // Tham số 2: Mảng dữ liệu  
        $this->view('home_index', $data);  
    }  
}
```

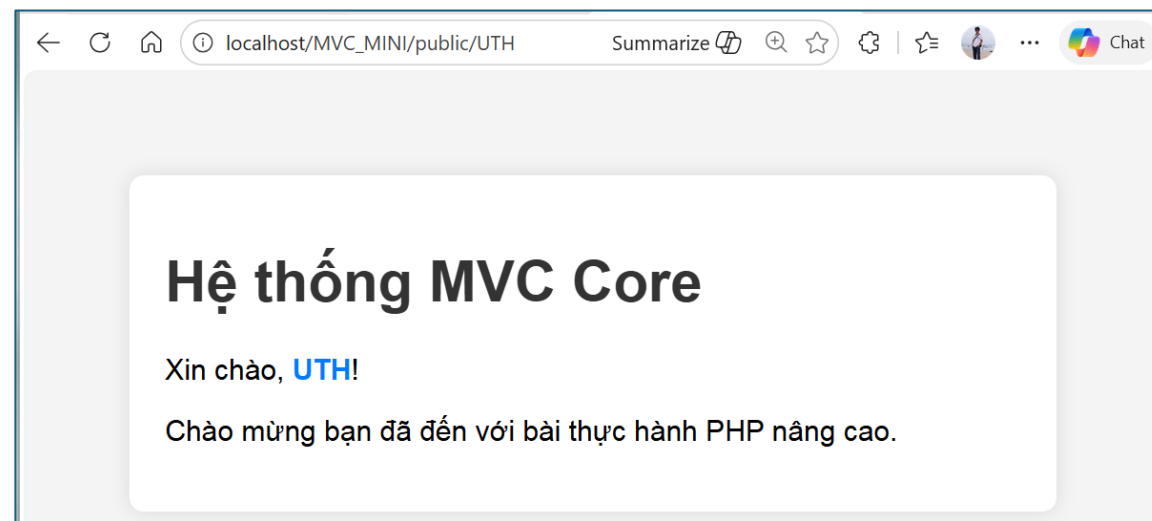
XÂY DỰNG BỘ KHUNG FRAMEWORK MINI

View mẫu mặc định (app/views/home_index.php)

```
<head>
  <meta charset="UTF-8">
  <title>Trang chủ MVC</title>
  <style>
    body { font-family: Arial, sans-serif; background-color: #f4f4f4; padding: 50px; }
    .container { background: white; padding: 20px; border-radius: 8px; box-shadow: 0 0
10px rgba(0,0,0,0.1); }
    h1 { color: #333; }
    .user-name { color: #007bff; font-weight: bold; }
  </style>
</head>
<body>
  <div class="container">
    <h1>Hệ thống MVC Core</h1>
    <p>Xin chào, <span class="user-name"><?php echo $data['name']; ?></span>!</p>
    <p>Chào mừng bạn đã đến với bài thực hành PHP nâng cao.</p>
  </div>
</body>
```

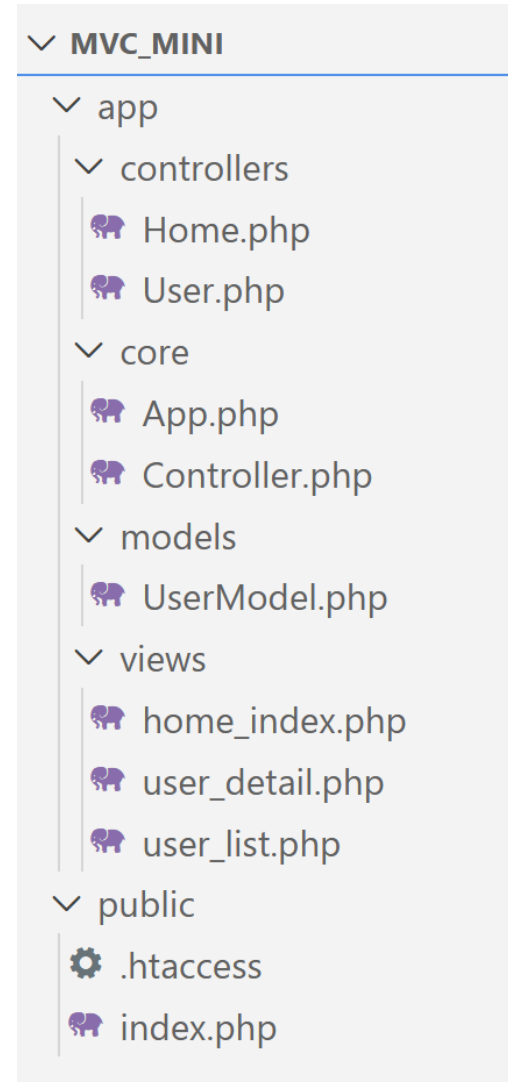
XÂY DỰNG BỘ KHUNG FRAMEWORK MINI

Giao diện trang mặc định



XÂY DỰNG BỘ KHUNG FRAMEWORK MINI

Xây dựng đủ các file từ Model, Controller đến View như mẫu bên



XÂY DỰNG BỘ KHUNG FRAMEWORK MINI

```
// app/controllers/User.php
class User extends Controller {
    public function index() {
        $userModel = $this->model('UserModel');
        $listUsers = $userModel->getAllUsers();
        $this->view('user_list', [
            'page_title' => 'Danh sách thành viên',
            'users' => $listUsers
        ]);
    }
    public function detail($id = 0) {
        $userModel = $this->model('UserModel');
        $user = $userModel->findUserId($id);
        if (!$user) { die("Người dùng không tồn tại!"); }
        $this->view('user_detail', [
            'page_title' => 'Hồ sơ cá nhân: ' . $user['name'],
            'user' => $user
        ]);
    }
}
```

▼ MVC_MINI

▼ app

▼ controllers

 Home.php

 **User.php**

▼ core

 App.php

 Controller.php

▼ models

 UserModel.php

▼ views

 home_index.php

 user_detail.php

 user_list.php

▼ public

 .htaccess

 index.php

XÂY DỰNG BỘ KHUNG FRAMEWORK MINI

```
// app/models/UserModel.php
class UserModel {
    // Giả lập dữ liệu trong Database
    private $users = [
        1 => [ 'id' => 1, 'name' => 'Nguyễn Văn A', 'role' => 'Admin',
              'phone' => '0901234567', 'address' => 'Hà Nội'],
        2 => [ 'id' => 2, 'name' => 'Trần Thị B', 'role' => 'Editor',
              'phone' => '0908887776', 'address' => 'Đà Nẵng'],
        3 => [ 'id' => 3, 'name' => 'Lê Văn C', 'role' => 'User',
              'phone' => '0905554443', 'address' => 'TP.HCM']
    ];

    public function getAllUsers() {
        return $this->users;
    }

    public function findUserById($id) {
        // Trong thực tế: SELECT * FROM users WHERE id = $id
        return $this->users[$id] ?? null;
    }
}
```

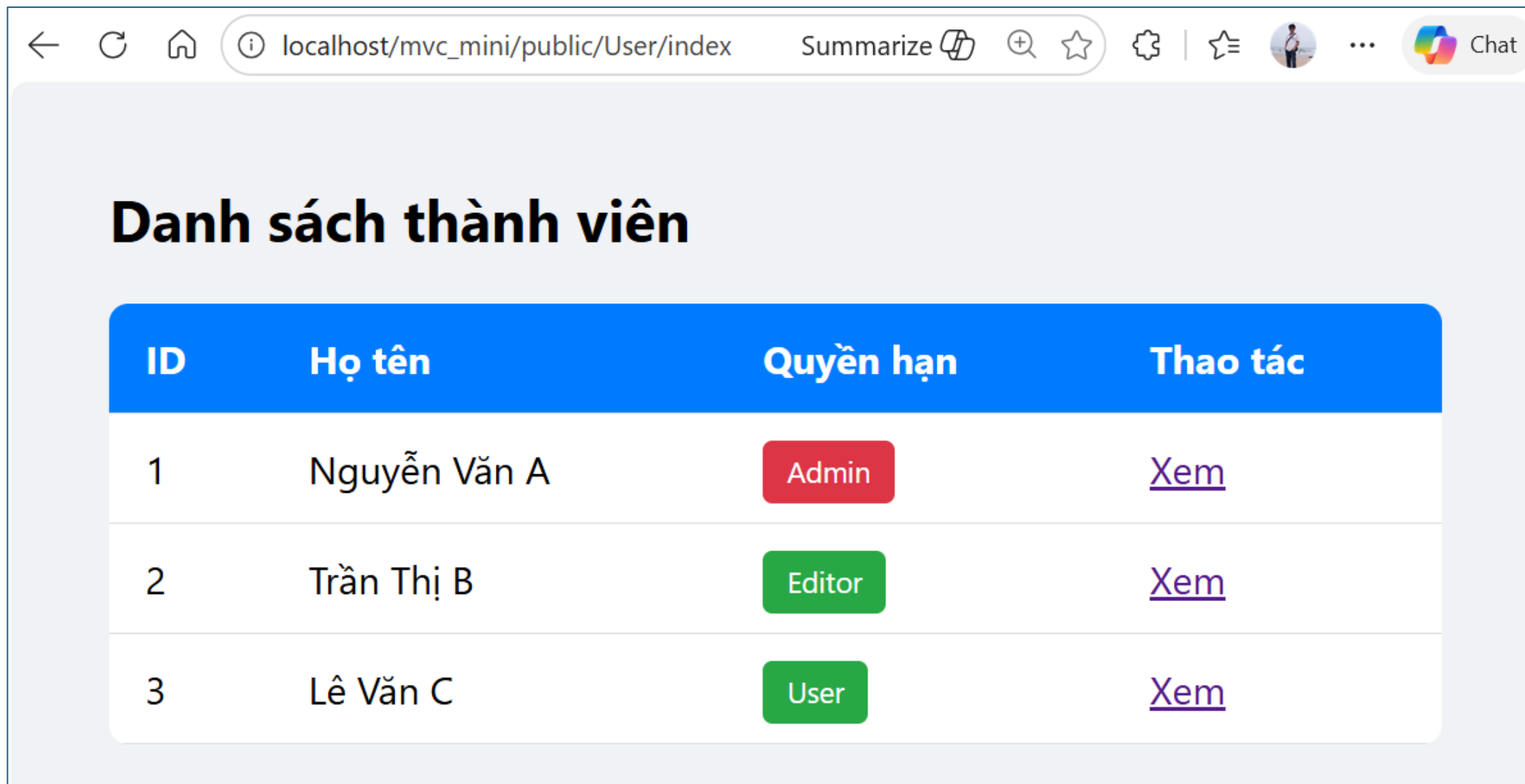
- ▼ MVC_MINI
 - ▼ app
 - ▼ controllers
 -  Home.php
 -  User.php
 - ▼ core
 -  App.php
 -  Controller.php
 - ▼ models
 -  UserModel.php
 - ▼ views
 -  home_index.php
 -  user_detail.php
 -  user_list.php
 - ▼ public
 -  .htaccess
 -  index.php

XÂY DỰNG BỘ KHUNG FRAMEWORK MINI

```
<h2><?= $data['page_title'] ?></h2>
<table>
  <thead>
    <tr><th>ID</th><th>Họ tên</th><th>Quyền hạn</th><th>Thao tác</th></tr>
  </thead>
  <tbody>
    <?php foreach ($data['users'] as $user): ?>
    <tr>
      <td><?= $user['id'] ?></td>
      <td><?= $user['name'] ?></td>
      <td>
        <span class="badge
          <?= strtolower($user['role']) == 'admin' ? 'admin':'user' ?>">
          <?= $user['role'] ?>
        </span>
      </td>
      <td>
        <a href="/mvc_mini/public/User/detail/<?= $user['id'] ?>">Xem</a>
      </td>
    </tr>
    <?php endforeach; ?>
  </tbody>
</table>
```

```
▼ MVC_MINI
  ▼ app
    ▼ controllers
      🐘 Home.php
      🐘 User.php
    ▼ core
      🐘 App.php
      🐘 Controller.php
    ▼ models
      🐘 UserModel.php
    ▼ views
      🐘 home_index.php
      🐘 user_detail.php
      🐘 user_list.php
  ▼ public
    ⚙ .htaccess
    🐘 index.php
```

XÂY DỰNG BỘ KHUNG FRAMEWORK MINI



ID	Họ tên	Quyền hạn	Thao tác
1	Nguyễn Văn A	Admin	Xem
2	Trần Thị B	Editor	Xem
3	Lê Văn C	User	Xem

XÂY DỰNG BỘ KHUNG FRAMEWORK MINI

```
<div class="card">
  <h2>Thông tin chi tiết</h2>
  <div class="info-group">
    <span class="label">Họ tên:</span>
    <span class="value"><?=$data['user']['name'] ?></span>
  </div>
  <div class="info-group">
    <span class="label">Số điện thoại:</span>
    <span class="value"><?=$data['user']['phone'] ?></span>
  </div>
  <div class="info-group">
    <span class="label">Địa chỉ:</span>
    <span class="value"><?=$data['user']['address'] ?></span>
  </div>
  <div class="info-group">
    <span class="label">Quyền hạn:</span>
    <span class="value">
      <strong><?=$data['user']['role'] ?></strong>
    </span>
  </div>
  <a href="/mvc_mini/public/User/index" class="back-btn">
    ← Quay lại danh sách
  </a>
</div>
```

▼ MVC_MINI

▼ app

▼ controllers

 Home.php

 User.php

▼ core

 App.php

 Controller.php

▼ models

 UserModel.php

▼ views

 home_index.php

 user_detail.php

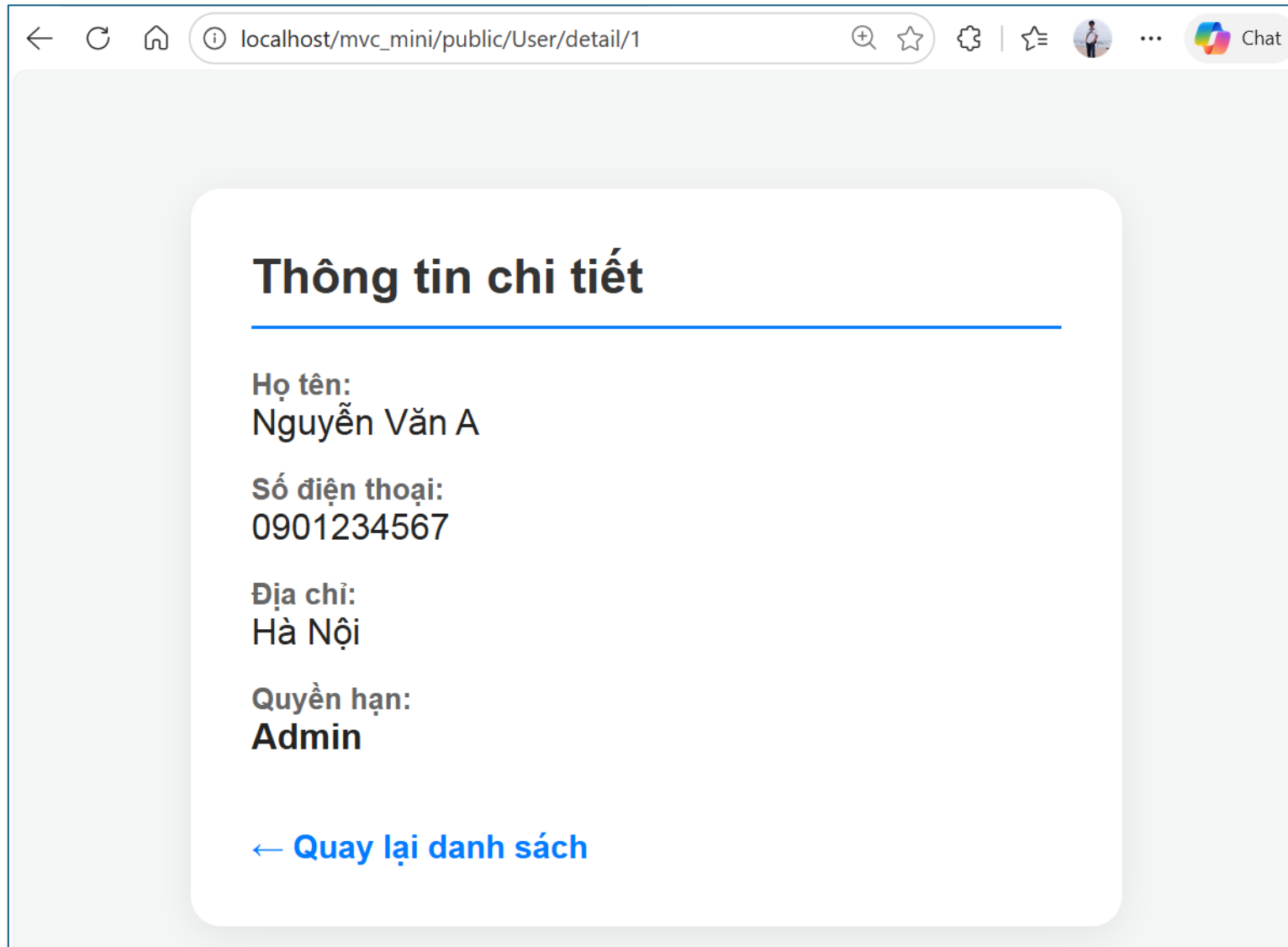
 user_list.php

▼ public

 .htaccess

 index.php

XÂY DỰNG BỘ KHUNG FRAMEWORK MINI



Trong MVC truyền thống, server sẽ nhận request, xử lý logic, render ra HTML hoàn chỉnh rồi gửi về cho client. Đó gọi là Server-Side Rendering (SSR).

Client-Side Rendering (CSR) trong mô hình MVC là cách mà bạn để trình duyệt (client) tự xử lý giao diện bằng JavaScript thay vì để server render HTML hoàn chỉnh.

Cơ chế hoạt động:

- Server chỉ trả về dữ liệu dạng JSON (thường qua API hoặc controller trả JSON).
- Giao diện được render bằng JavaScript trên trình duyệt.
- Các framework như React, Angular, Vue, Knockout, jQuery sẽ đảm nhiệm việc dựng UI.

CLIENT-SIDE RENDERING (CSR)

Bạn vẫn dùng MVC, nhưng:

Thành phần	Vai trò khi dùng CSR
Model	Cung cấp dữ liệu (thường trả về JSON)
Controller	Trả về JSON thay vì view
View	Thường là một trang HTML rỗng + JS để render UI
Client (JS)	Render giao diện, gọi API, cập nhật DOM

CLIENT-SIDE RENDERING (CSR)

Ví dụ: Controller PHP (products.php) trả về JSON. View HTML+JS render UI (index.html)

products.php

```
header('Content-Type: application/json');

// truy vấn DB

$products = [
    ["id" => 1, "name" => "iPhone 15"],
    ["id" => 2, "name" => "Samsung Galaxy S24"],
    ["id" => 3, "name" => "Xiaomi 14"]
];

echo json_encode($products);
```

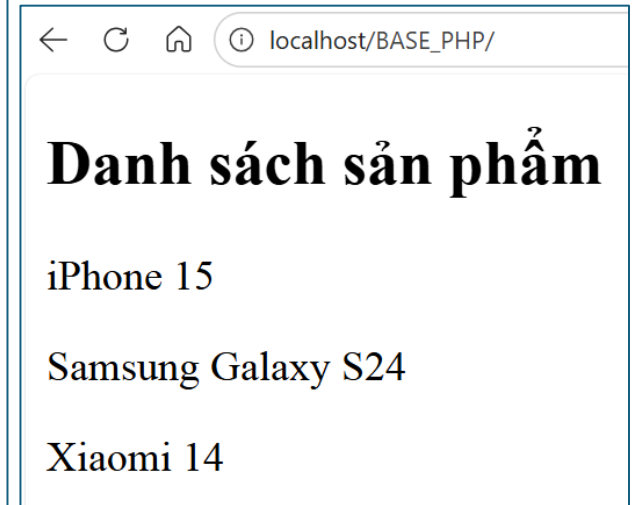
CLIENT-SIDE RENDERING (CSR)

Ví dụ: Controller PHP (products.php) trả về JSON. View HTML+JS render UI (index.html)

index.html

```
<body>
<h2>Danh sách sản phẩm</h2>
<div id="product-list"></div>

<script>
fetch('products.php')
  .then(response => response.json())
  .then(data => {
    const container = document.getElementById('product-list');
    container.innerHTML = data
      .map(p => `<p>${p.name}</p>`)
      .join('');
  });
</script>
</body>
```



Khi nào nên dùng CSR trong MVC

- Bạn muốn UI tương tác mạnh, cập nhật liên tục mà không reload trang
- Bạn xây dựng SPA (Single Page Application)
- Bạn muốn tách biệt rõ ràng giữa frontend và backend
- Bạn dùng các framework JS hiện đại

Ưu điểm

- UI mượt mà, trải nghiệm tốt
- Giảm tải cho server (server chỉ trả JSON)
- Tái sử dụng API cho mobile app

Nhược điểm

- SEO kém hơn SSR (trừ khi dùng SSR của React/Vue)
- Thời gian tải trang đầu tiên lâu hơn
- Cần nhiều JavaScript hơn

TÀI LIỆU THAM KHẢO

1. Trang chủ PHP: <https://www.php.net/>
2. PHP Tutorial: <https://www.w3schools.com/php/default.asp>
3. SQL Tutorial: <https://www.w3schools.com/sql/default.asp>
4. Regex: <https://regex101.com/>

THỰC HÀNH TỔNG HỢP

Thực hiện hoàn thiện website của nhóm.

Các nội dung lưu ý:

- Trang khách hàng
- Trang quản trị
- Hoàn thiện kết nối database và xử lý CRUD
- Kiểm thử toàn bộ tính năng của website
- Chuẩn bị demo và thuyết trình
-

Thực hiện hoàn thiện website của nhóm theo mô hình MVC.



THANKS

